

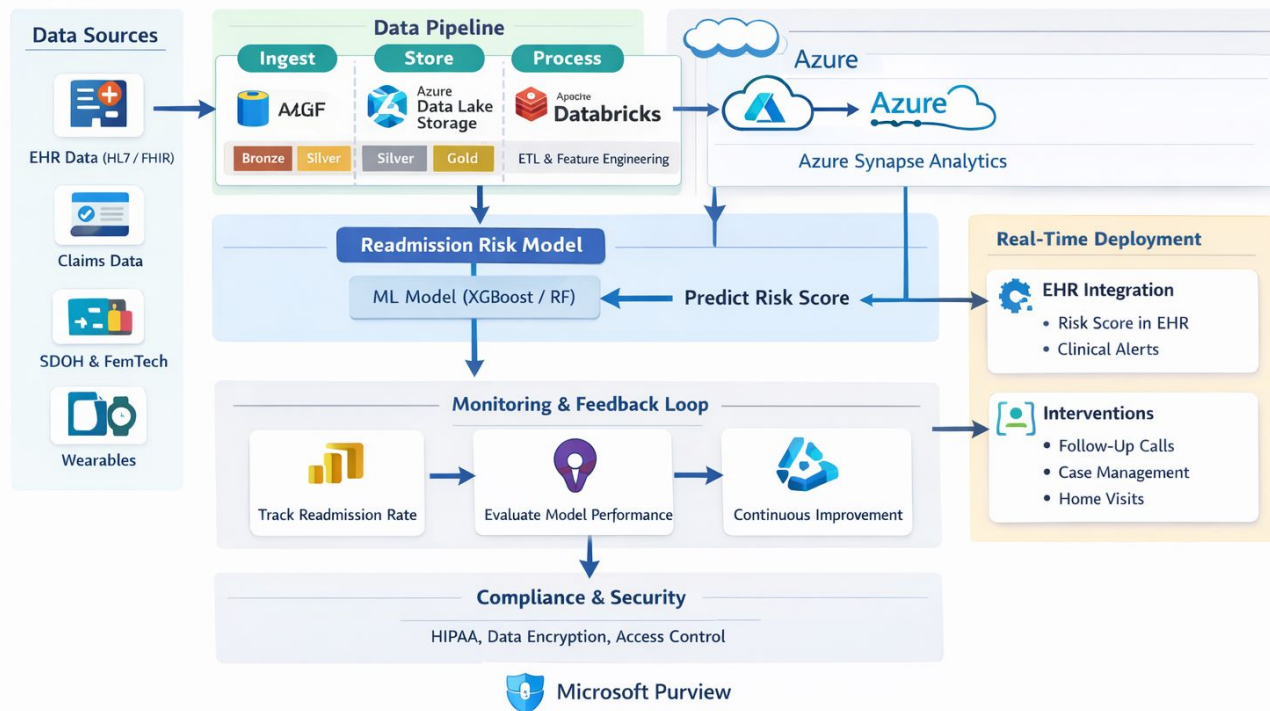
Diabetes 30-Day Readmission Prediction

Infrastructure Design & Clinical
Deployment Strategy

Xiao Hong Chen, Anthony Mapuyan, Chenkun Xiang, Joe Li



30-Day Readmission Reduction Workflow



Data Sources

Clinical / EHR Data

- ❑ Patient Demographics (age, gender)
- ❑ Admission & discharge info
- ❑ Diagnosis codes (ICD)
- ❑ Lab results (A1C, glucose levels)
- ❑ Medications (insulin, changes during stay)

Social Determinants of Health (SDOH)

- ❑ Income level (proxy via ZIP code)
- ❑ Access to transportation
- ❑ Housing stability
- ❑ Food insecurity

Claims & Utilization Data

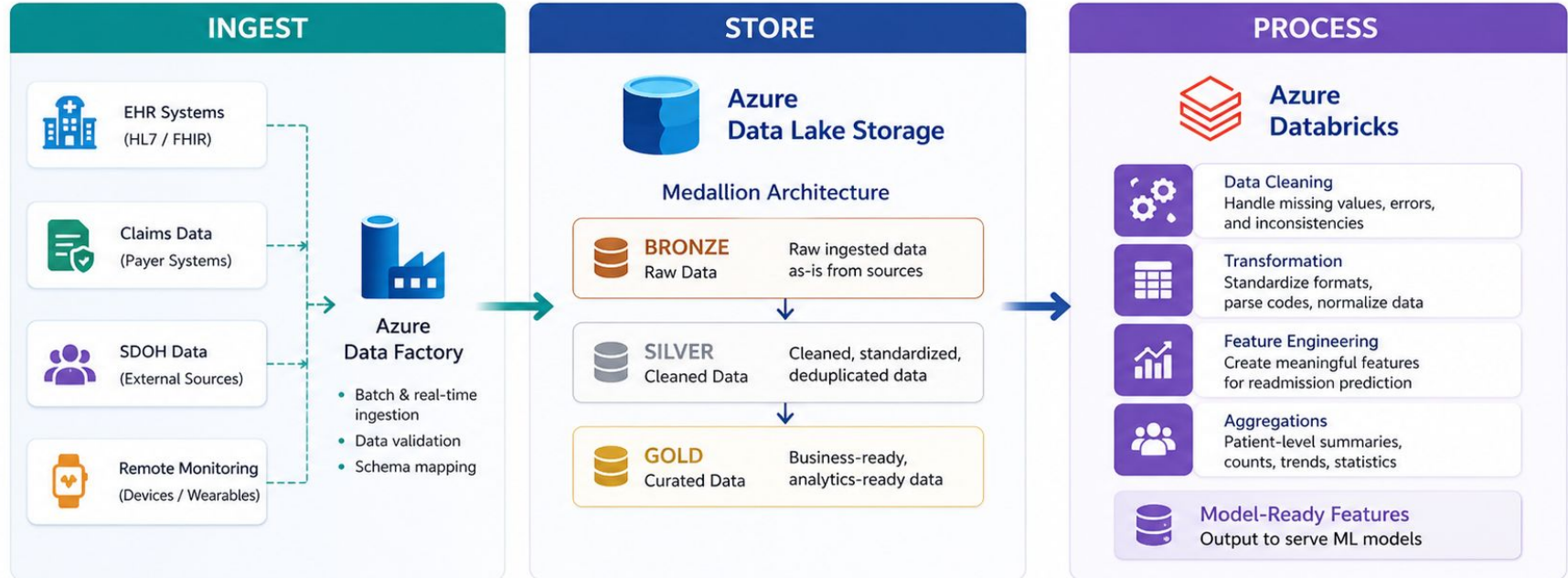
- ❑ Number of prior admissions
- ❑ Emergency visits
- ❑ Insurance type
- ❑ Length of stay trends

Remote Monitoring / Wearables

- ❑ Glucose monitoring devices
- ❑ Activity levels
- ❑ Heart rate trends

Data Pipeline

Ingest → Store → Process : Transforming raw healthcare data into model-ready features



Secure • Scalable • Governed (Azure Security, Role-based Access, Monitoring)

Azure Synapse Analytics

Unified analytics service to integrate, analyze, and visualize data at scale



AZURE SYNAPSE ANALYTICS

CORE CAPABILITIES



SQL Analytics

Run fast, serverless SQL queries on large datasets (Data Warehousing)



Big Data Analytics

Analyze large-scale data directly from the data lake



Data Integration

Integrate and combine data from multiple sources



BI & Visualization

Built-in visualizations and seamless integration with Power BI

OUTPUTS / INSIGHTS



Analytics & Reporting

Generate reports on readmission trends, risk scores, and outcomes



Business Insights

Identify high-risk patient groups and key drivers



Performance Monitoring

Track model performance and intervention effectiveness



Actionable Insights

Enable data-driven decisions to reduce 30-day readmissions



Connect to data in Data Lake and other sources



Query and process data using SQL at scale



Transform and combine data for analysis



Visualize and analyze to generate insights

Readmission Risk Model

Predict and score patients at risk of 30-day readmission

READMISSION RISK MODEL

KEY CAPABILITIES



Feature Utilization

Uses engineered features from clinical, utilization, demographic, and SDOH data to capture patient risk factors.



ML Algorithms

Applies advanced machine learning algorithms to learn patterns and predict readmission risk.



Risk Stratification

Categorizes patients based on predicted risk score:

- Low Risk: < 33%
- Medium Risk: 33% – 66%
- High Risk: 66% – 100%

OUTPUTS & INSIGHTS



Readmission Risk Score

Predicts the probability of 30-day readmission for each patient.



High-Risk Patient List

Identifies patients at high risk for proactive outreach and care interventions.



Model Performance

Monitors model accuracy, precision, recall, and AUC to ensure reliable predictions.



Actionable Insights

Enables care teams to take timely actions and reduce 30-day readmissions.



Input Data

Processed & engineered features from Databricks



Model Training

Train and validate the model on historical data



Risk Prediction

Generate readmission risk scores for patients



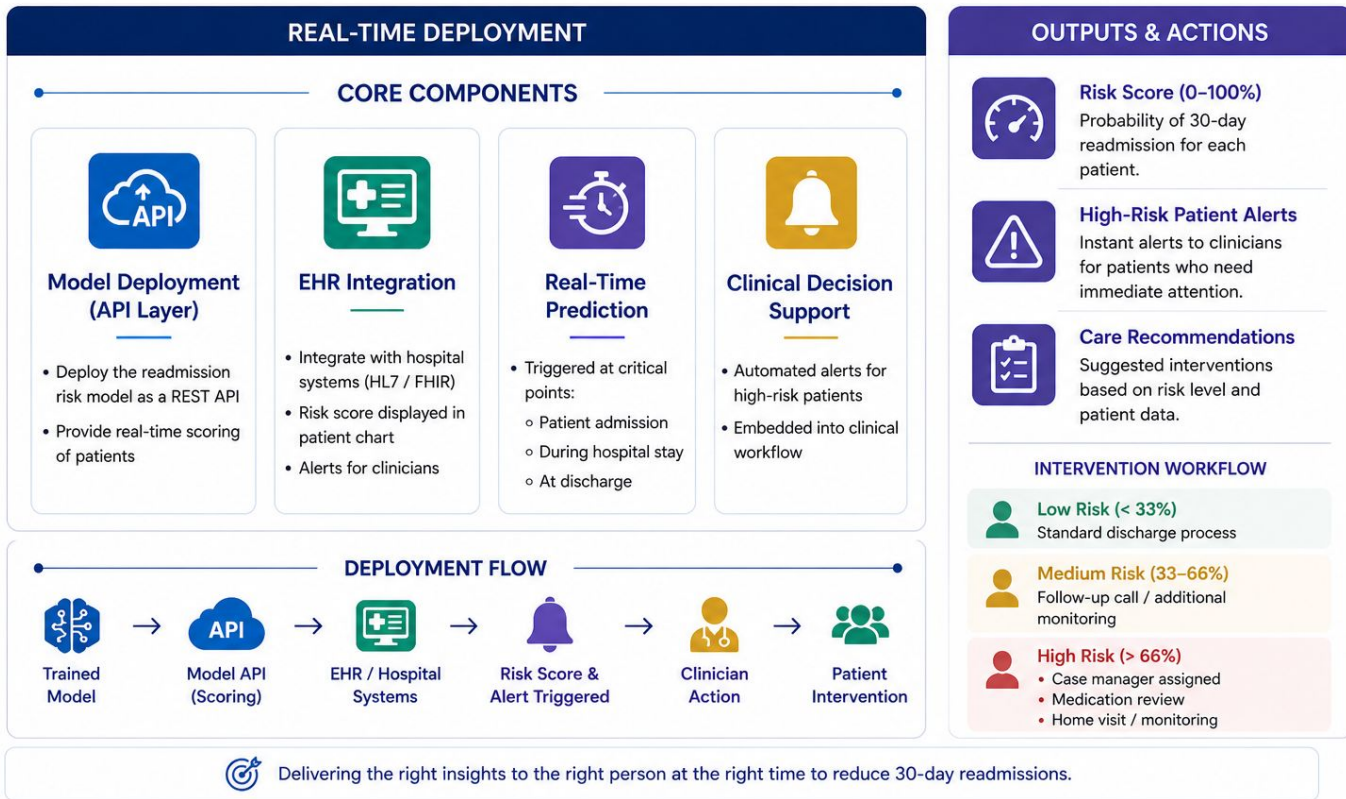
Insights & Actions

Deliver insights to care teams and applications



Real-Time Deployment

Deliver risk predictions to clinicians in real time to enable timely interventions





Monitoring & Feedback Loop / Compliance & Security

Continuously monitor model performance, ensure compliance, and protect patient data

MONITORING & FEEDBACK LOOP

MONITORING AREAS



Model Performance

- Track key metrics to ensure model accuracy
- AUC / ROC, Precision, Recall



Data Drift Detection

- Monitor data changes that may impact performance
- Feature, population and concept drift



Outcome Monitoring

- Track 30-day readmission rate
- Monitor intervention uptake and outcomes



Alerts & System Monitoring

- Monitor system health & API availability
- Track alert success rate



Model Retraining

- Retrain model with new data and outcomes
- Scheduled retraining

FEEDBACK LOOP PROCESS



1. Collect Data

Collect new patient data and outcomes



2. Analyze & Evaluate

Evaluate performance and identify areas for improvement



3. Improve Model

Refine features and retrain model



4. Redeploy

Deploy updated model for real-time use



5. Measure Impact

Measure impact on readmissions and patient outcomes

COMPLIANCE & SECURITY

DATA PROTECTION & PRIVACY



HIPAA Compliance

Ensure PHI is handled in compliance with HIPAA regulations.



Data Encryption

Encrypt data in transit (TLS 1.2+) and at rest (AES-256).



Access Control

Role-based access control (RBAC) and least privilege principles.



Audit & Logging

Maintain audit logs for data access, model predictions, and system activities.

SECURE ARCHITECTURE (AZURE)



Azure Key Vault

Manage secrets and keys securely



Private Endpoints

Secure connectivity to Azure services



Encryption at Rest

Protect data in storage



Network Security Groups

Control inbound and outbound traffic



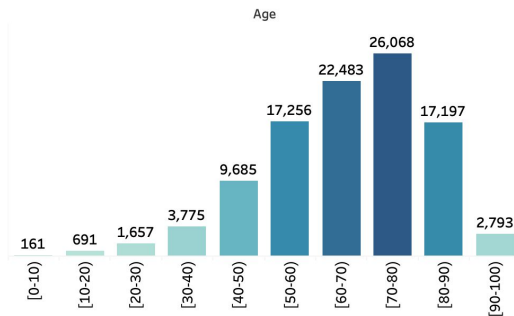
Goal: Continuously learn and improve to drive better outcomes and reduce 30-day readmissions.

Relevant Data Everyone Should know

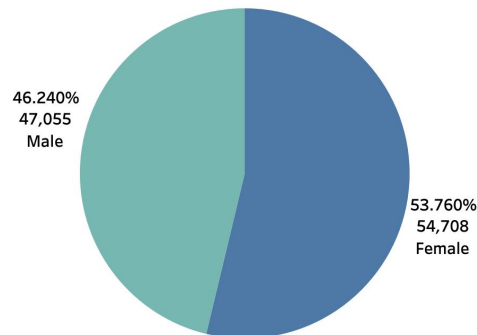
100,000 diabetic hospital encounters across a diverse patient population

Gender ■ Female ■ Male

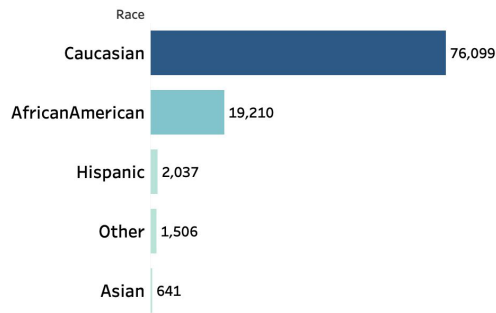
Age Distribution of Diabetic Patients



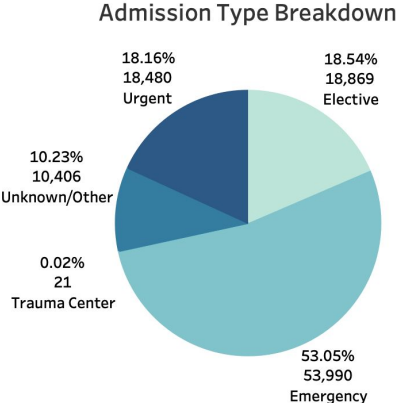
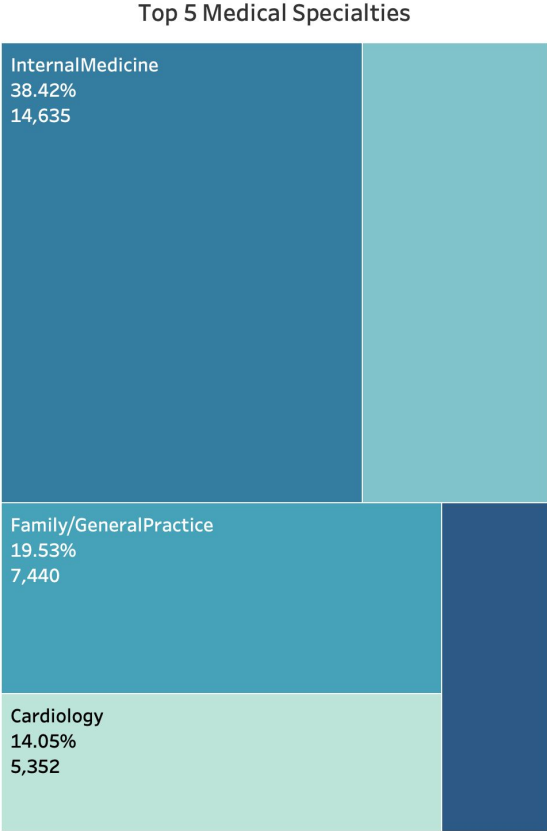
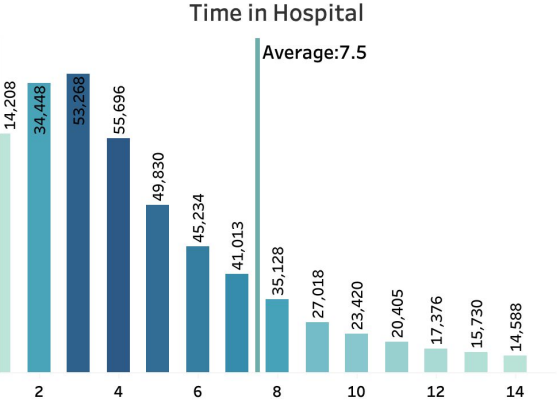
Gender Distribution of Diabetic Patients



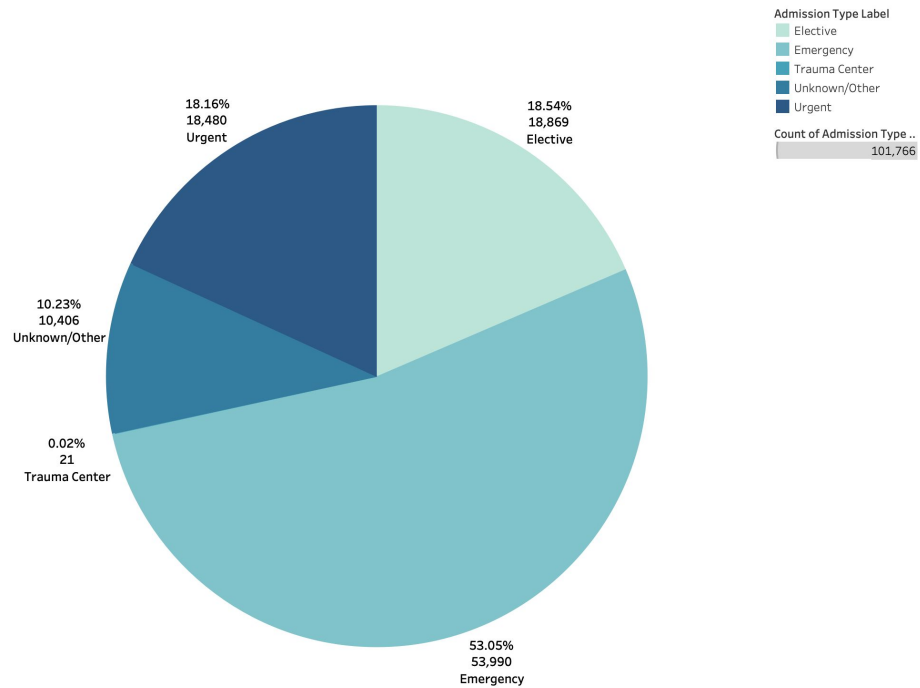
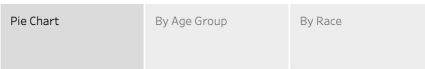
Race Distribution of Diabetic Patients



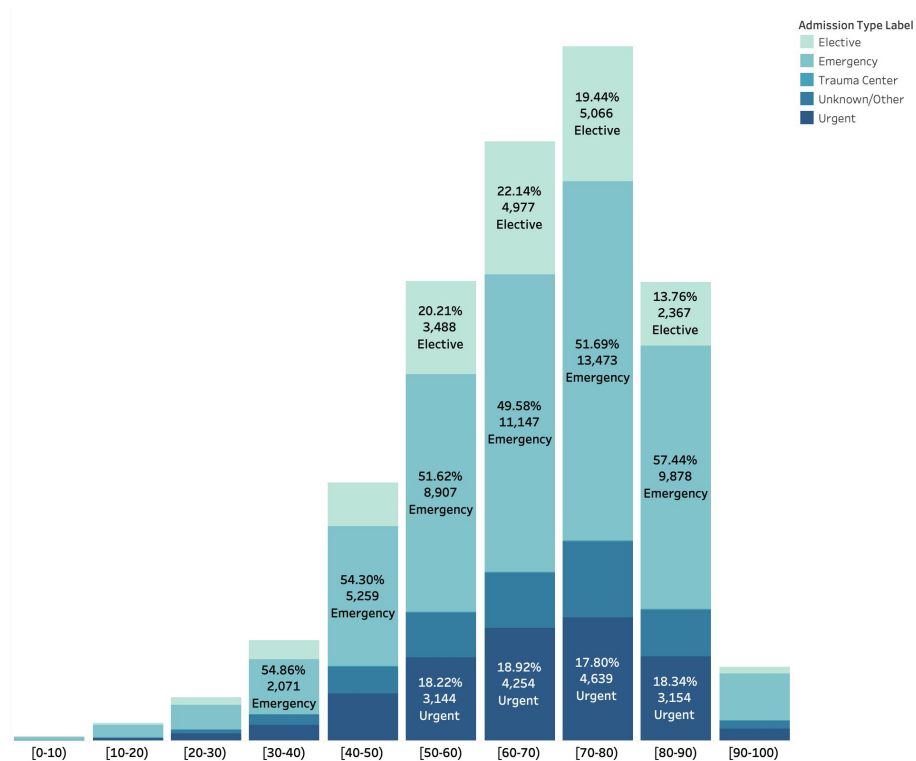
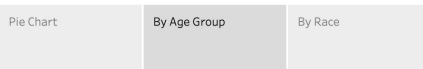
Most patients arrive as emergencies and stay an average of 7.5 days



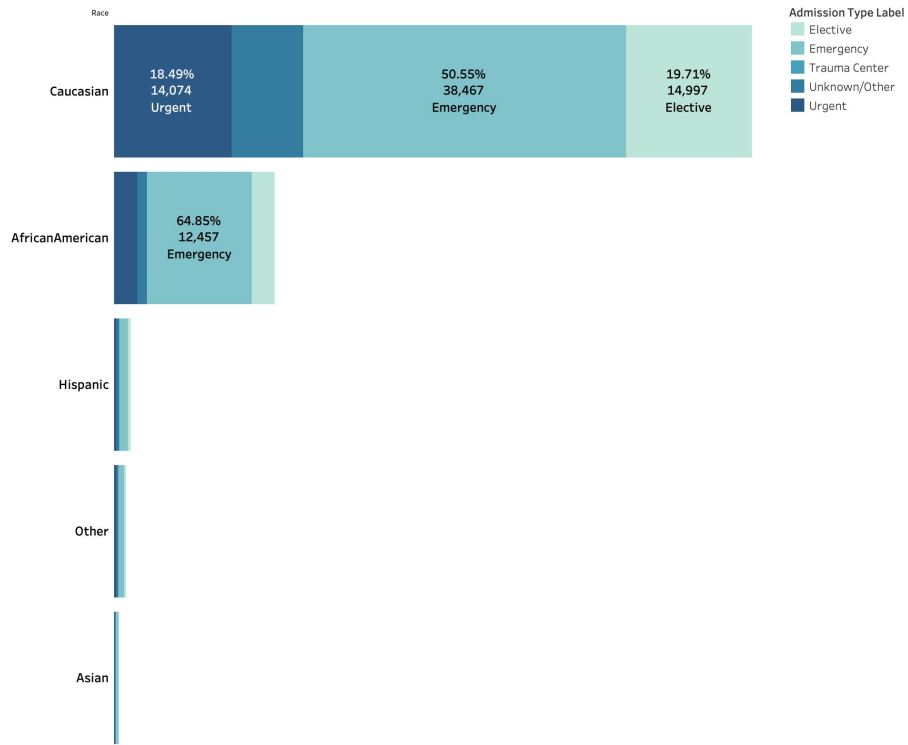
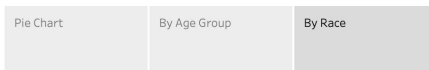
Admission Type



Admission Type



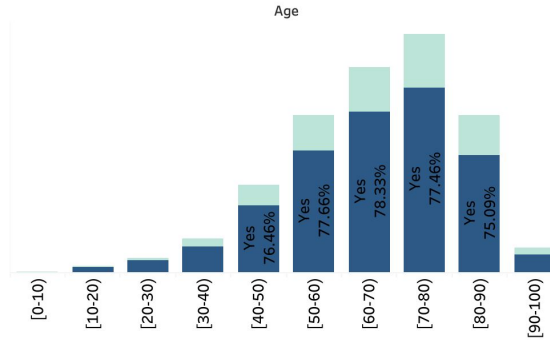
Admission Type



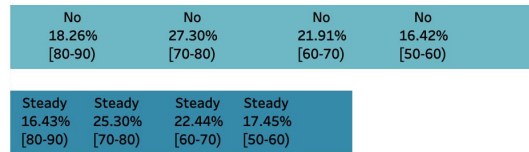
77% are on diabetes medication but nearly half receive no insulin

Age [0-10] [10-20] [20-30] [30-40] [40-50] [50-60] [60-70] [70-80]

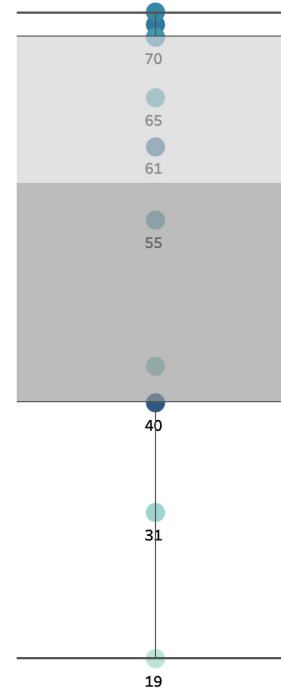
Diabetes Medication Usage - Age



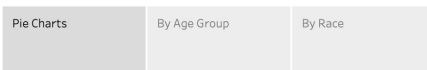
Insulin Usage Breakdown - Age



Number of Medications- Age



Diabetes Medication

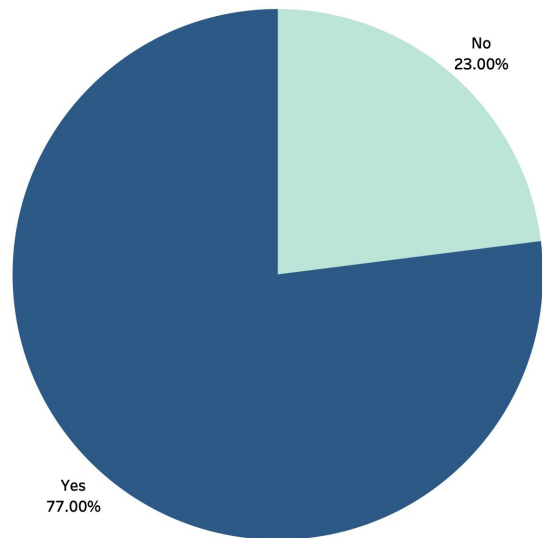


Diabetes Med

No
Yes

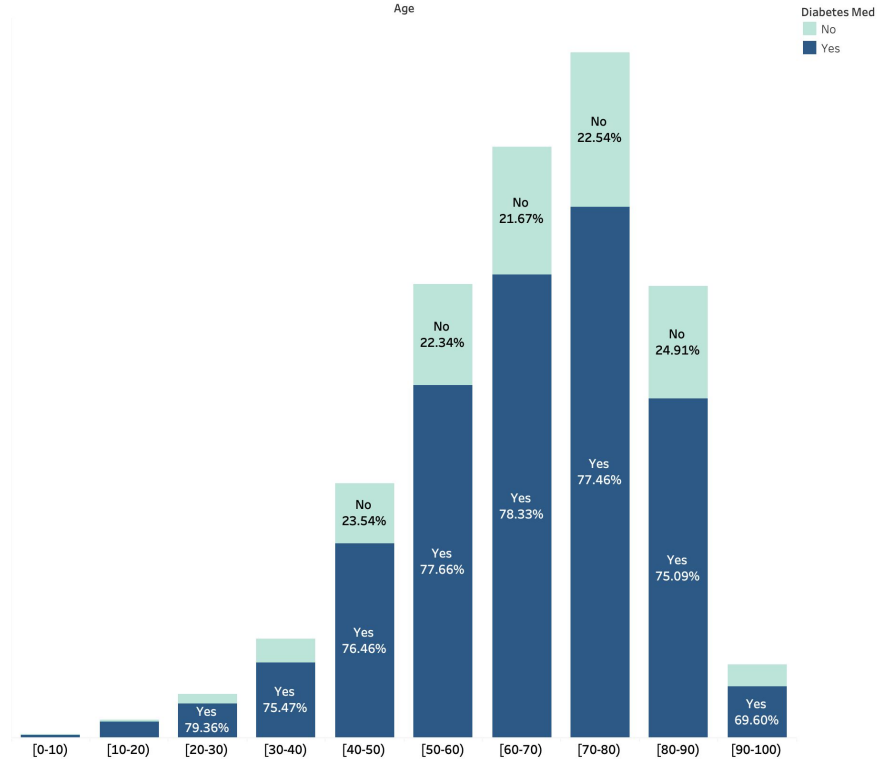
Count of diabetic_data.csv

101,766

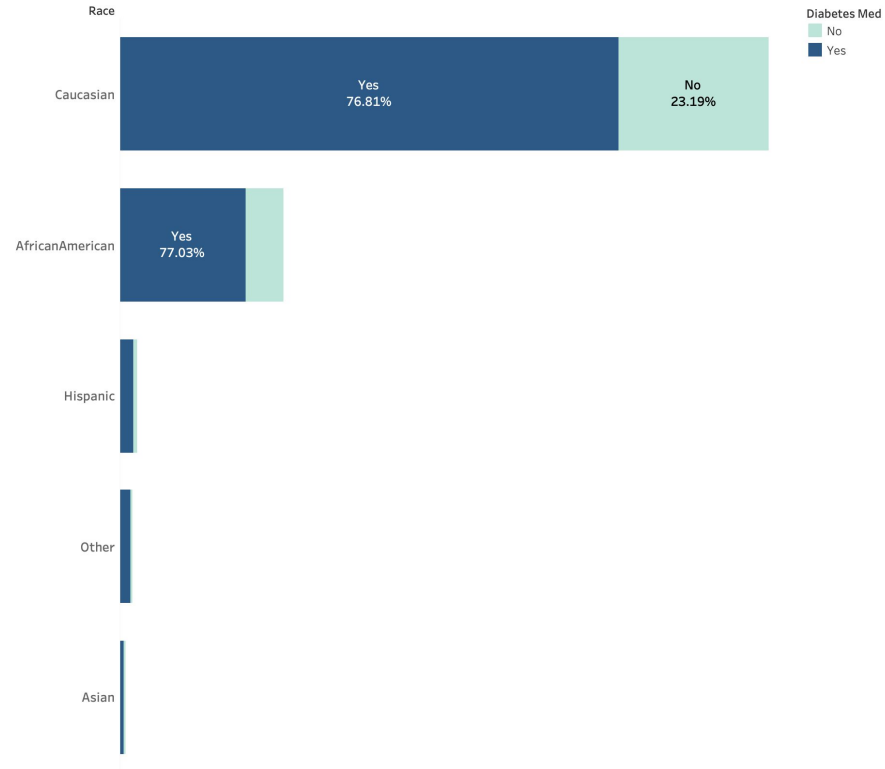
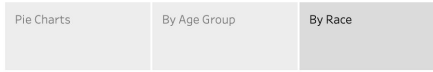


Diabetes Medication

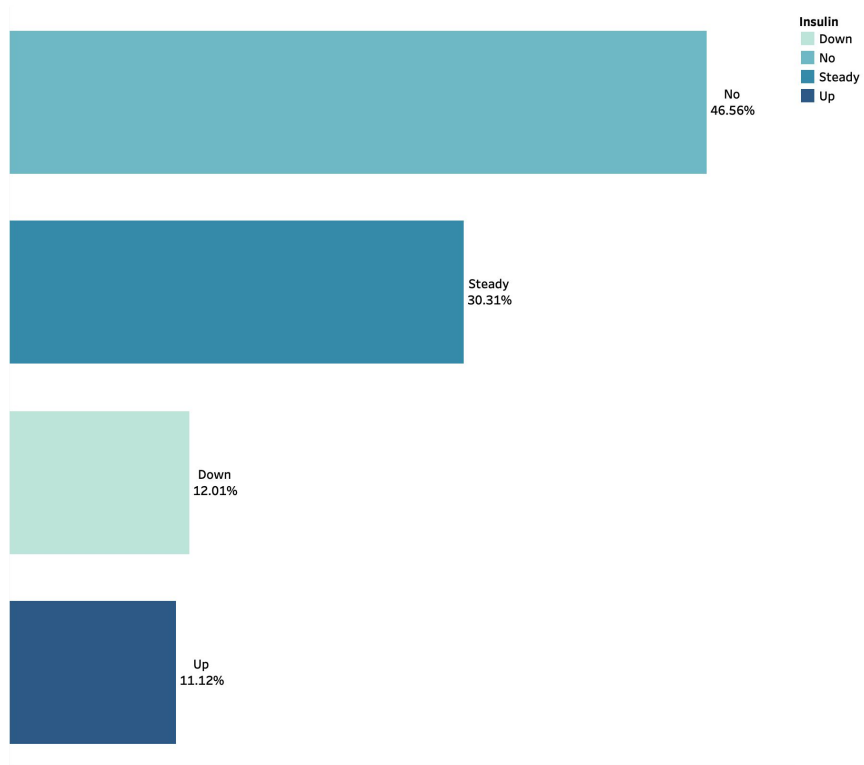
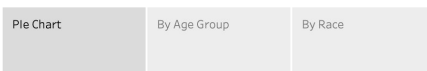
Pie Charts	By Age Group	By Race
------------	--------------	---------



Diabetes Medication



Insulin Usage

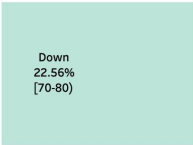
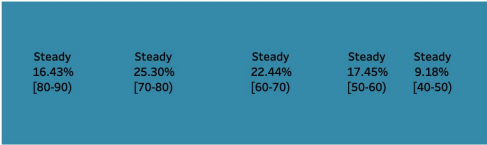


Insulin Usage

Pie Chart

By Age Group

By Race

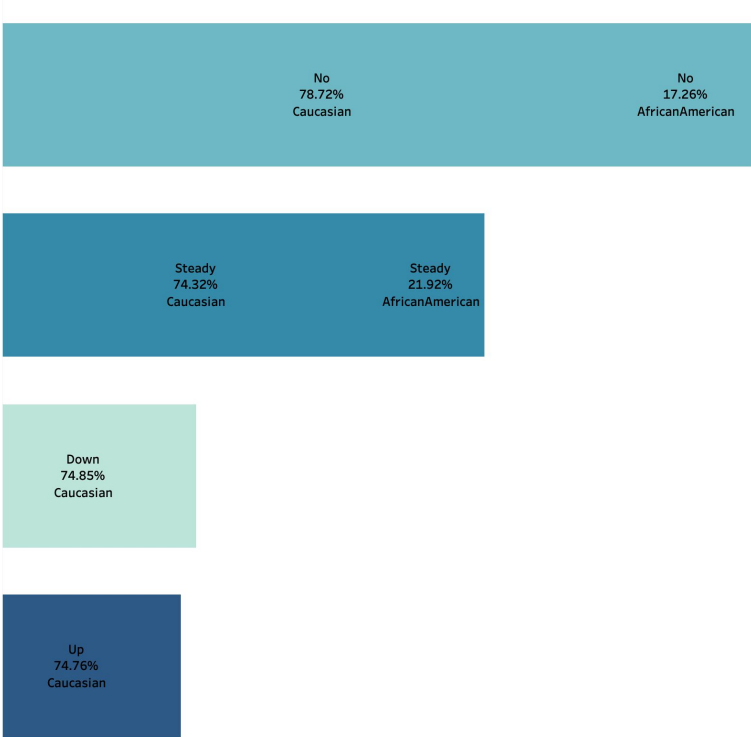


Insulin

- Down
- No
- Steady
- Up

Insulin Usage

Pie Chart	By Age Group	By Race
-----------	--------------	---------



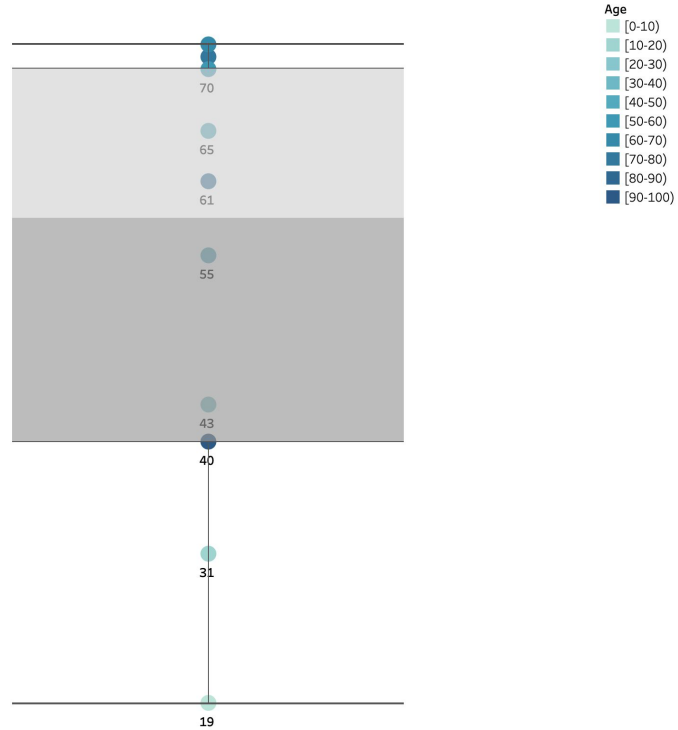
Insulin

- Down
- No
- Steady
- Up

Number of Medications

By Age Group

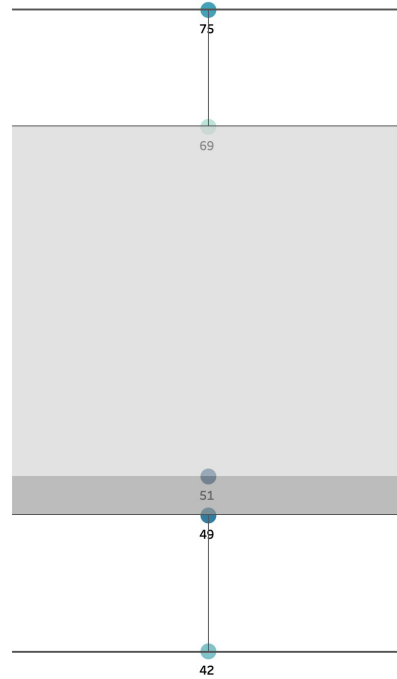
By Race



Number of Medications

By Age Group

By Race



Race

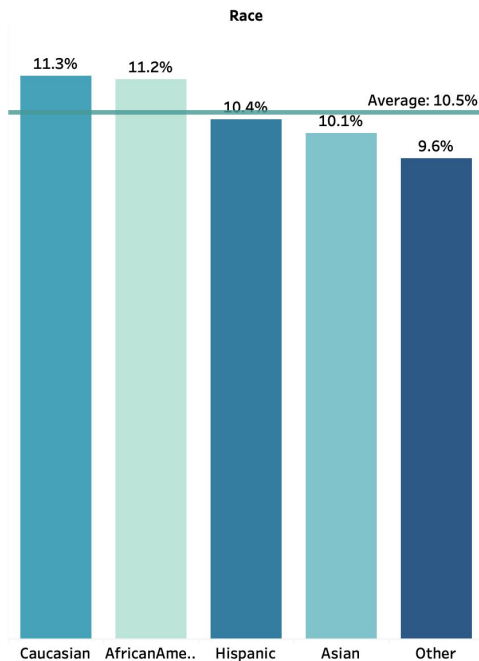
- African American
- Asian
- Caucasian
- Hispanic
- Other

1 in 9 patients is readmitted within 30 days

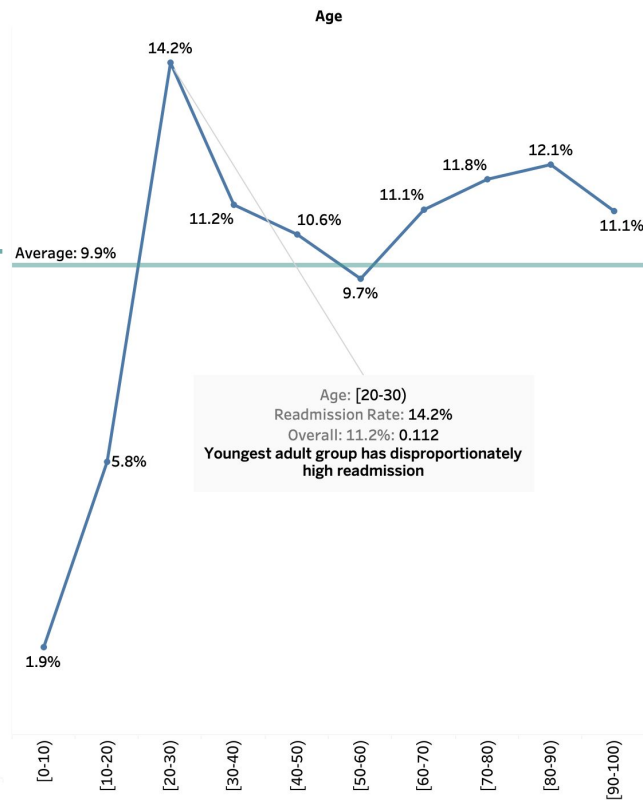
Overall Readmission Rate

11.2%

Readmission Rate by Race



Readmission Rate by Age Group



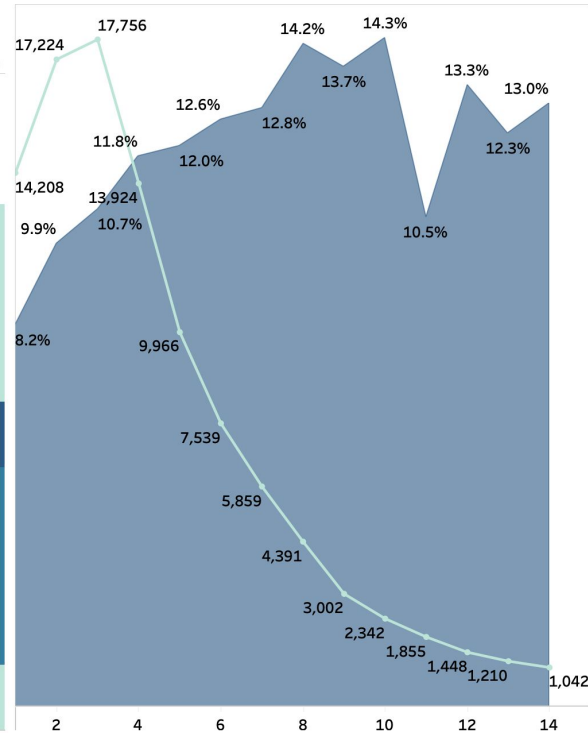
Sicker, more complex patients face significantly higher readmission risk



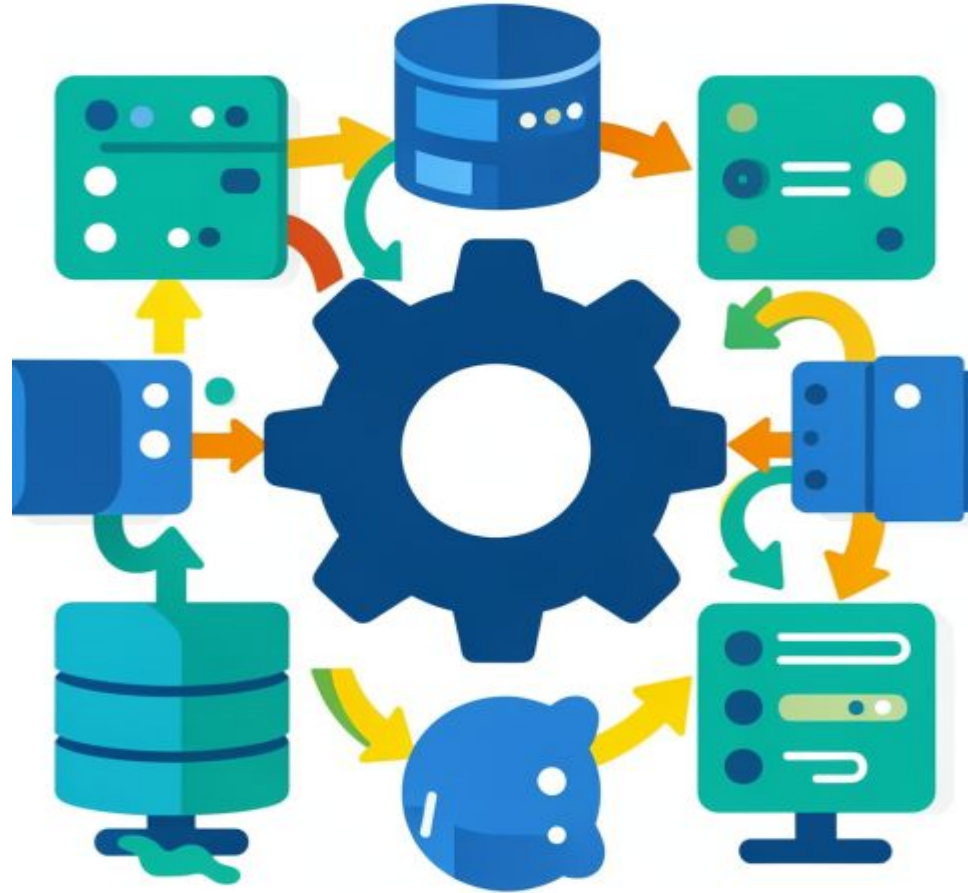
Readmission Rate vs. Number of Diagnoses

Age	Diagnosis Complexity			
	Low (1-3)	Medium (4-7)	High (8-10)	Very High (11+)
[0-10)	0.0160	0.0000	0.1111	
[10-20)	0.0528	0.0676	0.0435	
[20-30)	0.1388	0.1251	0.1762	0.0000
[30-40)	0.0822	0.1015	0.1347	0.0000
[40-50)	0.0668	0.0881	0.1279	0.0000
[50-60)	0.0513	0.0764	0.1156	0.2000
[60-70)	0.0617	0.0959	0.1231	0.1579
[70-80)	0.0761	0.1087	0.1239	0.1538
[80-90)	0.0868	0.1166	0.1233	0.1250
[90-100)	0.1212	0.1092	0.1117	0.0000

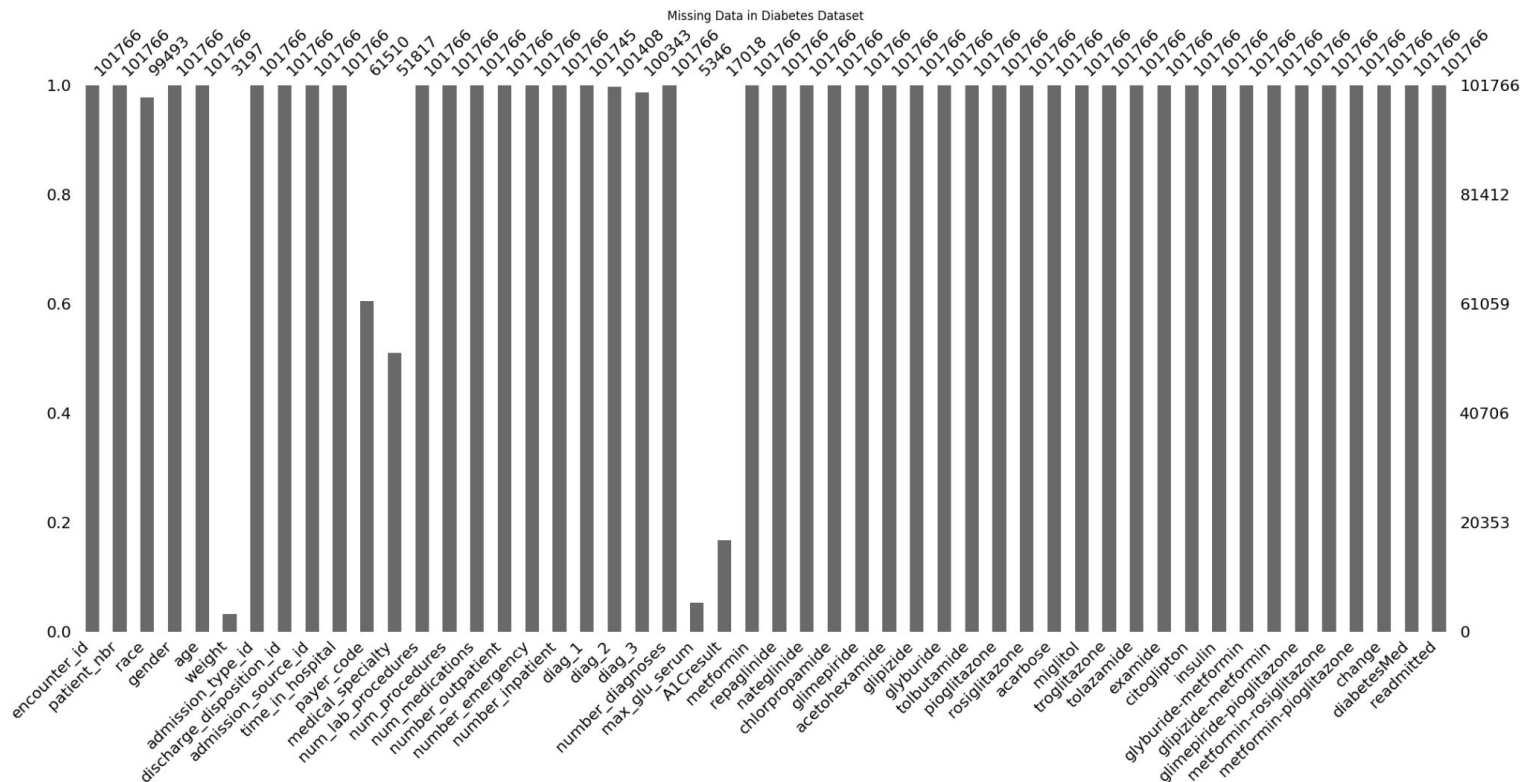
Readmission Rate vs. Time in Hospital



Data Cleaning (Baseline Pipeline)



Overview of Data Values



Original Dataset:

- 101,766 patients
- 50 columns

"?" values were masking missing data - replaced with NaN before running msno.bar for missing data graph

Data Cleaning (Baseline Pipeline)

Columns	Missing Percentage
weight	96.8585%
max_glu_serum	94.7468%
A1Cresult	83.2773%
medical_specialty	49.0822%
payer_code	39.5574%
race	02.2336%
diag_3	01.3983%
diag_2	00.3518%
diag_1	00.0206%

Removed:

- ❑ weight
- ❑ max_glu_serum
- ❑ A1Cresult
- ❑ encounter_id
- ❑ patient_nbr

Kept with “Unknown” fill:

- ❑ medical_specialty
- ❑ diag_1/2/3

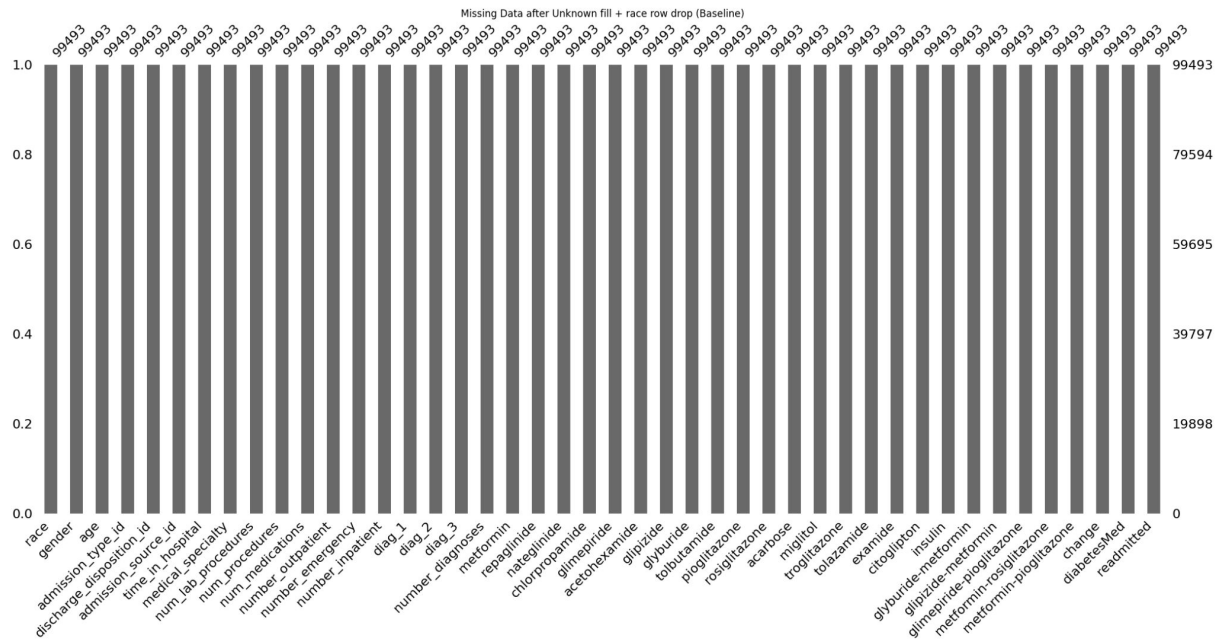
Dropped (baseline decision):

- ❑ dropped the missing 2.23%(2,273 rows) race rows
- ❑ dropped payer_code entirely for now (40% missing)

ETL Transformation : Missing Values

Invalid Admission ID remapping

- ❑ admission_type {5, 6, 8}
- ❑ discharge_disposition {18, 25, 26}
- ❑ admission_source {9, 15, 17, 20, 21}
- ❑ remapped as "Unknown" as per IDS_mapping



ETL Transformation : Diag_1/2/3

ICD-9 Grouping

- ❑ diag_1/2/3 have hundreds of unique ICD-9 codes
- ❑ grouped by ICD-9 numeric range into clinical buckets (Circulatory, Endocrine, Respiratory, etc)
- ❑ Empty cells filled in as “Unknown”
- ❑ Results: 3 columns → 18 meaningful categories each

```
def classify_icd9(code_str):
    if pd.isna(code_str) or code_str == 'Unknown':
        return 'Unknown'
    code_str = str(code_str).strip()
    if code_str.startswith('V'): return 'Supplementary'
    if code_str.startswith('E'): return 'External_Causes'
    try:
        code_num = float(code_str)
    except ValueError:
        return 'Unknown'
    if 1 <= code_num < 140: return 'Infectious_Parasitic'
    elif 140 <= code_num < 240: return 'Neoplasms'
    elif 240 <= code_num < 280: return 'Endocrine'
    elif 280 <= code_num < 290: return 'Blood'
    elif 290 <= code_num < 320: return 'Mental'
    elif 320 <= code_num < 390: return 'Nervous_System'
    elif 390 <= code_num < 460: return 'Circulatory'
    elif 460 <= code_num < 520: return 'Respiratory'
    elif 520 <= code_num < 580: return 'Digestive'
    elif 580 <= code_num < 630: return 'Genitourinary'
    elif 630 <= code_num < 680: return 'Pregnancy_Childbirth'
    elif 680 <= code_num < 710: return 'Skin'
    elif 710 <= code_num < 740: return 'Musculoskeletal'
    elif 740 <= code_num < 760: return 'Congenital'
    elif 760 <= code_num < 780: return 'Perinatal'
    elif 780 <= code_num < 800: return 'Symptoms_Signs'
    elif 800 <= code_num < 1000: return 'Injury_Poisoning'
    else: return 'Unknown'

for col in ['diag_1', 'diag_2', 'diag_3']:
    diabetes[col] = diabetes[col].apply(classify_icd9)

print('diag1-3 grouped categories:')
print(diabetes[['diag_1', 'diag_2', 'diag_3']].head(10))
```


ETL Transformation: Age Encoding

Age Ordinal Encoding

- ❑ age brackets like [60-70) → 0-9 integer scales
- ❑ preserves sequential relationships

```
age_mapping = {  
    '[0-10)': 0,  
    '[10-20)': 1,  
    '[20-30)': 2,  
    '[30-40)': 3,  
    '[40-50)': 4,  
    '[50-60)': 5,  
    '[60-70)': 6,  
    '[70-80)': 7,  
    '[80-90)': 8,  
    '[90-100)': 9  
}  
diabetes['age'] = diabetes['age'].map(age_mapping)  
diabetes.head(10)
```

Variable

Target Variable Recoding

- ❑ **<30 days → READMITTED**
- ❑ >30 days → NOT readmitted
- ❑ No revisit → NOT readmitted
- ❑ Only early readmission rate is important

```
readmit_map = {'<30': 1, '>30': 0, 'NO': 0, 1: 1, 0: 0, '1': 1, '0': 0}  
diabetes['readmitted'] = diabetes['readmitted'].map(readmit_map)
```

Cleaned data summary:

Shape: (99493, 44)

Columns: 44

Missing values: 0

Target distribution:

readmitted

0 88324

1 11169

Name: count, dtype: int64

Split

- ❑ Split before scaling or encoding to prevent data leakage
- ❑ 70% training / 15% validation / 15% test
- ❑ stratified on target variable
 - ❑ ensures 11.2% readmission rate preserved on every split

```
# 1) 70% train, 30% temp
train_df, temp_df = train_test_split(
    diabetes,
    test_size=0.30,
    random_state=42,
    stratify=diabetes["readmitted"]
)

# 2) Split temp into 15% valid, 15% test
valid_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=42,
    stratify=temp_df["readmitted"]
)

# Clean indices
train_df = train_df.reset_index(drop=True)
valid_df = valid_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

# Quick check of class balance
print("Row counts:")
print(f"train: {len(train_df)} valid: {len(valid_df)} test: {len(test_df)}")

print("\nClass ratio (readmitted=1) by split:")
print("total :", diabetes["readmitted"].mean())
print("train :", train_df["readmitted"].mean())
print("valid :", valid_df["readmitted"].mean())
print("test :", test_df["readmitted"].mean())
```

```
Row counts:
train: 69645 valid: 14924 test: 14924
```

```
Class ratio (readmitted=1) by split:
total : 0.11225915391032534
train : 0.1122500753822959
valid : 0.11230233181452694
test : 0.11223532564995979
```

ETL Transformation:

Encode Medication, Change, DiabetesMed

Medication columns

- ❑ ordinal coded each medication
- ❑ No=0, Steady=1, Up=2, Down=3

```
med_map = {'No': 0, 'Steady': 1, 'Up': 2, 'Down': 3}
med_cols = [
    'metformin', 'repaglinide', 'nateglinide', 'chlorpropamide',
    'glimepiride', 'acetohexamide', 'glipizide', 'glyburide',
    'tolbutamide', 'pioglitazone', 'rosiglitazone', 'acarbose',
    'miglitol', 'troglitazone', 'tolazamide', 'examide',
    'citoglipton', 'insulin', 'glyburide-metformin',
    'glipizide-metformin', 'glimepiride-pioglitazone',
    'metformin-rosiglitazone', 'metformin-pioglitazone'
]
```

Binary Encoded change & diabetesMed Column

- ❑ change → binary: medication changed during stay = 1, else = 0
- ❑ diabetesMed → binary: on diabetes medication = 1, else = 0

```
# — ENCODE CHANGE AND DIABETESMED (binary) —
if 'change' in x_tr.columns:
    x_tr['change'] = (x_tr['change'] == 'Ch').astype(int)
    x_val['change'] = (x_val['change'] == 'Ch').astype(int)

if 'diabetesMed' in x_tr.columns:
    x_tr['diabetesMed'] = (x_tr['diabetesMed'] == 'Yes').astype(int)
    x_val['diabetesMed'] = (x_val['diabetesMed'] == 'Yes').astype(int)
```

ETL Transformation: One-Hot Encode Nominal Columns

Encoding Categories with No Natural Order

- ❑ 9 columns have no numerical order
 - ❑ ace, gender, medical_specialty, diag_1/2/3, admission type/source/disposition
- ❑ Applied one-hot-encoding
- ❑ Fit encoder on training data only, then apply to validation & test

```
# onehotencode for nominal categorical columns
nominal_cols = [
    'race',
    'gender',
    'medical_specialty',
    'diag_1',
    'diag_2',
    'diag_3',
    'admission_type_id',
    'discharge_disposition_id',
    'admission_source_id',
]

# Fit OHE on x_train only - handle_unknown='ignore' silently zeros out
# any category seen in valid/test but not in train
ohe = OneHotEncoder(drop='first', sparse_output=False, handle_unknown='ignore')
ohe.fit(x_train[nominal_cols])

# Generate column names from the fitted encoder
ohe_feature_names = ohe.get_feature_names_out(nominal_cols)

# Transform all three splits using the train-fitted encoder
def apply_ohe(df, ohe, nominal_cols, continuous_cols, ohe_feature_names):
    ohe_array = ohe.transform(df[nominal_cols])
    ohe_df = pd.DataFrame(ohe_array, columns=ohe_feature_names, index=df.index)
    return pd.concat([df[continuous_cols].reset_index(drop=True), ohe_df.reset_index(drop=True)], axis=1)

x_train = apply_ohe(x_train, ohe, nominal_cols, continuous_cols, ohe_feature_names)
x_valid = apply_ohe(x_valid, ohe, nominal_cols, continuous_cols, ohe_feature_names)
x_test = apply_ohe(x_test, ohe, nominal_cols, continuous_cols, ohe_feature_names)

Encoded shapes:
x_train: (69645, 178)
x_valid: (14924, 178)
x_test : (14924, 178)

Column alignment check passed.
```

Columns

Standardize Continuous Columns

- ❑ 8 continuous columns with different scales
 - ❑ lab procedures ,time in hospital, etc
- ❑ Applied StandardScaler to bring all continuous values to the same scale
- ❑ Scaler fit on training data only → applied to validation & test

```
# standardscaler for continuous features
scaler = StandardScaler()
continuous_cols = ['time_in_hospital', 'num_lab_procedures', 'num_procedures',
                  'num_medications', 'number_outpatient', 'number_emergency',
                  'number_inpatient', 'number_diagnoses']

x_train[continuous_cols] = scaler.fit_transform(x_train[continuous_cols])
x_valid[continuous_cols] = scaler.transform(x_valid[continuous_cols])
x_test[continuous_cols] = scaler.transform(x_test[continuous_cols])

x_train.head()
```

✓ 0.0s

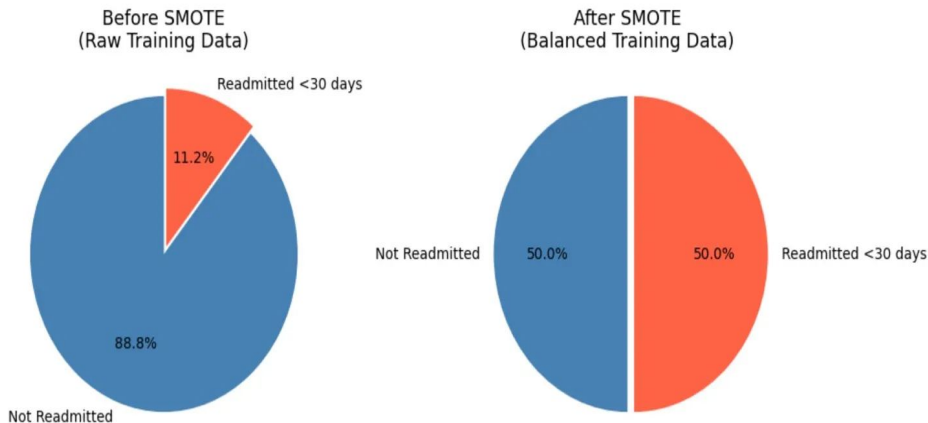
	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	number_diagnoses
0	-0.132201	0.354840	0.975385	0.246172	-0.293894	-0.225759	0.275002	-0.751767
1	1.880880	0.506701	-0.199602	0.617585	1.289784	-0.225759	-0.504285	0.808050
2	-1.138742	-0.353847	-0.787095	-1.363282	0.497945	0.884385	-0.504285	0.288111
3	-0.132201	0.000496	-0.787095	0.122368	-0.293894	-0.225759	-0.504285	0.808050
4	-1.138742	-0.353847	-0.199602	-0.125241	-0.293894	-0.225759	-0.504285	-0.231828

ETL Transformation: Fix Class Imbalance with SMOTE

SMOTE

- ❑ 88.8% not readmitted vs 11.2% readmitted
- ❑ generate synthetic examples of the minority class to bring training data to 50/50

Class Imbalance: Before vs After SMOTE
SMOTE generates synthetic readmission cases so the model learns both classes equally



Before SMOTE - Not Readmitted: 61,861 | Readmitted: 7,823
After SMOTE - Not Readmitted: 61,861 | Readmitted: 61,861

Cleaning Completed

	Before	After
Rows	101,766	99,493
Columns	50	178 (OHE)
Missing Values	Thousands	0
Class Balance	88/12	50/50 (SMOTE)
Target Variable	3 outcomes	Binary 0/1

Three Dataframe Comparison



Model Configuration Comparison

Columns	Missing Percentage
weight	96.8585%
max_glu_serum	94.7468%
A1Cresult	83.2773%
medical_specialty	49.0822%
payer_code	39.5574%
race	02.2336%
diag_3	01.3983%
diag_2	00.3518%
diag_1	00.0206%

THE MISSING DATA CHALLENGE

Two variables had significant missing values:
- payer_code (payment method): 40% missing
- race: 2% missing

The Question:

Should we:

- (a) Drop these columns entirely?
- (b) Keep them and fill missing values as "Unknown"?

This matters because both could be clinically meaningful:

- Payer type affects care access → readmission risk
- Race correlates with social determinants of health

Configuration Testing

```
# — Configuration 1: BASELINE (current approach) —
df_baseline = diabetes_original.copy()
df_baseline = df_baseline.drop(columns=['payer_code'])
df_baseline = df_baseline.dropna(subset=['race'])
print(f"Config 1 - Baseline: {df_baseline.shape}")

# — Configuration 2: WITH PAYER_CODE —
df_with_payer = diabetes_original.copy()
df_with_payer['payer_code'] = df_with_payer['payer_code'].fillna('Unknown')
df_with_payer = df_with_payer.dropna(subset=['race'])
print(f"Config 2 - With payer_code: {df_with_payer.shape}")

# — Configuration 3: WITH RACE UNKNOWN —
df_with_race = diabetes_original.copy()
df_with_race = df_with_race.drop(columns=['payer_code'])
df_with_race['race'] = df_with_race['race'].fillna('Unknown')
print(f"Config 3 - With race Unknown: {df_with_race.shape}")
```

```
Config 1 - Baseline: (99493, 44)
Config 2 - With payer_code: (99493, 45)
Config 3 - With race Unknown: (101766, 44)
```

THREE DATA CONFIGURATIONS

Configuration 1: BASELINE

- ✗ Drop payer_code entirely
 - ✗ Drop rows with missing race
- Result: 99,493 patients

Configuration 2: WITH PAYER_CODE

- ✓ Keep payer_code, fill "Unknown"
 - ✗ Drop rows with missing race
- Result: 99,493 patients

Configuration 3: WITH RACE UNKNOWN

- ✗ Drop payer_code entirely
 - ✓ Keep race, fill "Unknown"
- Result: 101,766 patients (kept all rows)

Testing Strategy

COMPARISON: 3 DATA CONFIGURATIONS x 3 MODELS							
Configuration	Model	AUC	Recall	Precision	Accuracy	Specificity	Best Threshold
Baseline (drop payer_code, drop race NaN)	Decision Tree	0.584939	0.448235	0.158150	0.670205	0.697972	0.296
Baseline (drop payer_code, drop race NaN)	Logistic Regression	0.605984	0.417714	0.173330	0.710984	0.748070	0.230
Baseline (drop payer_code, drop race NaN)	Random Forest	0.626116	0.570317	0.161334	0.618945	0.625095	0.467
With payer_code (Unknown)	Decision Tree	0.584959	0.450030	0.153375	0.659389	0.660512	0.335
With payer_code (Unknown)	Logistic Regression	0.610115	0.432675	0.176213	0.709237	0.744211	0.213
With payer_code (Unknown)	Random Forest	0.624227	0.594853	0.158129	0.598992	0.599516	0.462
With race (Unknown)	Decision Tree	0.598138	0.456151	0.155842	0.663580	0.666938	0.321
With race (Unknown)	Logistic Regression	0.602071	0.566804	0.148405	0.588703	0.591453	0.167
With race (Unknown)	Random Forest	0.619273	0.483225	0.164661	0.668768	0.692075	0.485

HOW WE TESTED

For each configuration, we trained 3 different models:

1. Logistic Regression (simple, interpretable)
2. Decision Tree (rule-based)
3. Random Forest (ensemble of trees)

All models:

- Trained on same 70% of data
- Validated on same 15% of data
- Used SMOTE to balance the minority class
- Measured on AUC and Recall

Winning Configuration

Configuration: WITH PAYER_CODE Model: RANDOM FOREST

Performance:

- AUC: 0.624
- Recall: 0.595 ← Highest of all combinations
- Precision: 0.178

What this means:

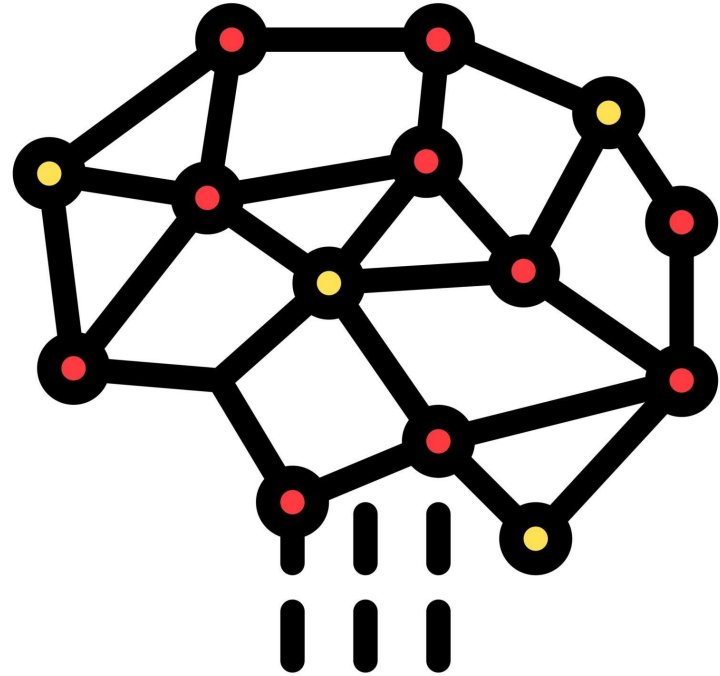
- ✓ We correctly identify 59.5% of patients who will be readmitted
- ✓ Compared to 0% if we did nothing

Clinical Impact:

Out of every 100 patients who will be readmitted:

- We catch 60 of them
- We can intervene before discharge

13 Machine Learning Model Selections



Model Selection - Finding the Best Algorithm

Standardize Continuous Columns

- ❑ df_with_payer selected as main dataset
- ❑ Tested 13 different ML models to find the best predictor of 30-day readmission
 - ❑ All trained on the same SMOTE - balanced 70% training datasets
 - ❑ All evaluated on same 15% validation set
 - ❑ Hypertuned models used RandomizedSearchCV/ GridSearchCV to optimize for Recall

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import SGDClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from catboost import CatBoostClassifier
from sklearn.metrics import make_scorer
from sklearn.metrics import (roc_auc_score, accuracy_score,
                             precision_score, f1_score, recall_score, roc_curve)
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
import xgboost as xgb
```

Measuring Performance

Metrics Tracked:

- ❑ AUC-ROC - overall discrimination ability
- ❑ **Recall** - % of actual readmissions caught
- ❑ Precision - % of predictions that were correct
- ❑ F1 Score - balance of recall and precision
- ❑ Accuracy
- ❑ Specificity

Recall First

- ❑ False negative = missed high-risk patient → readmitted
- ❑ False positive = flag a patient who wasn't readmitted → extra follow-up call
- ❑ Missing readmission far more costly than an unnecessary phone call
- ❑ "Better safe than sorry"

Baseline Model Results

Logistic Regression

```
lr = LogisticRegression(random_state=42, solver='newton-cg', max_iter=1000)
lr.fit(x_train_res, y_train_res)

lr_preds = lr.predict_proba(x_valid)[: , 1]

fpr, tpr, thresholds = roc_curve(y_valid, lr_preds)
lr_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {lr_best_thresh:.3f}")
print('Metrics for Validation data - Logistic Regression:')

lr_valid_auc, lr_valid_accuracy, lr_valid_recall, \
| lr_valid_precision, lr_valid_f1, lr_valid_specificity = print_report(y_valid, lr_preds, lr_best_thresh)
```

✓ 0.6s

Optimal threshold: 0.213
Metrics for Validation data - Logistic Regression:
AUC:0.610
Accuracy:0.709
recall:0.433
precision:0.176
F1 Score:0.250
specificity:0.744
prevalence:0.112

KNN Model

```
knn = KNeighborsClassifier(n_neighbors=100, n_jobs=-1)
knn.fit(x_train_res, y_train_res)

knn_preds = knn.predict_proba(x_valid)[: , 1]

fpr, tpr, thresholds = roc_curve(y_valid, knn_preds)
knn_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {knn_best_thresh:.3f}")
print('Metrics for Validation data - KNN:')

knn_valid_auc, knn_valid_accuracy, knn_valid_recall, \
| knn_valid_precision, knn_valid_f1, knn_valid_specificity = print_report(y_valid, knn_preds, knn_best_thresh)
```

✓ 3.9s

Optimal threshold: 0.310
Metrics for Validation data - KNN:
AUC:0.615
Accuracy:0.495
recall:0.679
precision:0.140
F1 Score:0.232
specificity:0.449
prevalence:0.112

Baseline Model Results

Linear SVC

```
lsvc_clf = LinearSVC(random_state=42, max_iter=1000)
lsvc_clf.fit(x_train_res, y_train_res)

lsvc_preds_raw = lsvc_clf.decision_function(x_valid)

# Normalize decision function scores to [0,1] for threshold + roc_curve
lsvc_scaler = MinMaxScaler()
lsvc_preds = lsvc_scaler.fit_transform(lsvc_preds_raw.reshape(-1, 1)).flatten()

fpr, tpr, thresholds = roc_curve(y_valid, lsvc_preds)
lsvc_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {lsvc_best_thresh:.3f}")
print('Metrics for Validation data - Linear SVC:')

lsvc_valid_auc, lsvc_valid_accuracy, lsvc_valid_recall, \
    lsvc_valid_precision, lsvc_valid_f1, lsvc_valid_specificity = print_report(y_valid, lsvc_preds, lsvc_best_thresh)
```

✓ 3.9s

Optimal threshold: 0.762
Metrics for Validation data - Linear SVC:
AUC:0.614
Accuracy:0.716
recall:0.431
precision:0.180
F1 Score:0.254
specificity:0.752
prevalence:0.112

Decision Tree

```
dc_clf = DecisionTreeClassifier(random_state=42, max_depth=10)
dc_clf.fit(x_train_res, y_train_res)

dc_preds = dc_clf.predict_proba(x_valid)[:, 1]

fpr, tpr, thresholds = roc_curve(y_valid, dc_preds)
dc_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {dc_best_thresh:.3f}")
print('Metrics for Validation data - Decision Tree:')

dc_valid_auc, dc_valid_accuracy, dc_valid_recall, \
    dc_valid_precision, dc_valid_f1, dc_valid_specificity = print_report(y_valid, dc_preds, dc_best_thresh)
```

✓ 1.0s

Optimal threshold: 0.335
Metrics for Validation data - Decision Tree:
AUC:0.585
Accuracy:0.659
recall:0.450
precision:0.153
F1 Score:0.229
specificity:0.661
prevalence:0.112

Baseline Model Results

Stochastic Gradient Descent Model

```
sgdc = SGDClassifier(loss='log_loss', alpha=0.1, random_state=42)
sgdc.fit(x_train_res, y_train_res)

sgd_preds = sgdc.predict_proba(x_valid)[:, 1]

fpr, tpr, thresholds = roc_curve(y_valid, sgd_preds)
sgd_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {sgd_best_thresh:.3f}")
print('Metrics for Validation data - SGD Classifier:')

sgdc_valid_auc, sgdc_valid_accuracy, sgdc_valid_recall, \
    sgdc_valid_precision, sgdc_valid_f1, sgdc_valid_specificity = print_report(y_valid, sgd_preds, sgd_best_thresh)
```

✓ 0.7s

Optimal threshold: 0.463
Metrics for Validation data - SGD Classifier:
AUC:0.620
Accuracy:0.606
recall:0.561
precision:0.155
F1 Score:0.243
specificity:0.612
prevalence:0.112

Random Forest

```
rf_clf = RandomForestClassifier(random_state=111, max_depth=6, n_estimators=300, n_jobs=-1)
rf_clf.fit(x_train_res, y_train_res)

rf_preds = rf_clf.predict_proba(x_valid)[:, 1]

fpr, tpr, thresholds = roc_curve(y_valid, rf_preds)
rf_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {rf_best_thresh:.3f}")
print('Metrics for Validation data - Random Forest:')

rf_valid_auc, rf_valid_accuracy, rf_valid_recall, \
    rf_valid_precision, rf_valid_f1, rf_valid_specificity = print_report(y_valid, rf_preds, rf_best_thresh)
```

✓ 4.4s

Optimal threshold: 0.475
Metrics for Validation data - Random Forest:
AUC:0.623
Accuracy:0.645
recall:0.524
precision:0.163
F1 Score:0.249
specificity:0.661
prevalence:0.112

Baseline Model Results

Gradient Boosting

```
gb_clf = GradientBoostingClassifier(n_estimators=100, criterion='friedman_mse',
                                   learning_rate=1.0, max_depth=3, random_state=42)
gb_clf.fit(x_train_res, y_train_res)

gb_preds = gb_clf.predict_proba(x_valid)[: , 1]

fpr, tpr, thresholds = roc_curve(y_valid, gb_preds)
gb_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {gb_best_thresh:.3f}")
print('Metrics for Validation data - Gradient Boosting:')

gb_valid_auc, gb_valid_accuracy, gb_valid_recall, \
gb_valid_precision, gb_valid_f1, gb_valid_specificity = print_report(y_valid, gb_preds, gb_best_thresh)
```

✓ 31.0s

Optimal threshold: 0.120
Metrics for Validation data - Gradient Boosting:
AUC:0.647
Accuracy:0.660
recall:0.536
precision:0.173
F1 Score:0.261
specificity:0.675
prevalence:0.112

XGBoost

```
xgb_clf = xgb.XGBClassifier(max_depth=3, learning_rate=1.0,
                             eval_metric='logloss', random_state=42)
xgb_clf.fit(x_train_res, y_train_res)

xgb_preds = xgb_clf.predict_proba(x_valid)[: , 1]

fpr, tpr, thresholds = roc_curve(y_valid, xgb_preds)
xgb_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {xgb_best_thresh:.3f}")
print('Metrics for Validation data - XGBoost:')

xgb_valid_auc, xgb_valid_accuracy, xgb_valid_recall, \
xgb_valid_precision, xgb_valid_f1, xgb_valid_specificity = print_report(y_valid, xgb_preds, xgb_best_thresh)
```

✓ 0.8s

Optimal threshold: 0.131
Metrics for Validation data - XGBoost:
AUC:0.649
Accuracy:0.676
recall:0.518
precision:0.177
F1 Score:0.264
specificity:0.696
prevalence:0.112

Baseline Model Results

CatBoost

```
catb = CatBoostClassifier(iterations=200, depth=3, learning_rate=1.0,  
                           random_state=111, verbose=0)  
catb.fit(x_train_res, y_train_res)  
  
catb_preds = catb.predict_proba(x_valid)[: , 1]  
  
fpr, tpr, thresholds = roc_curve(y_valid, catb_preds)  
catb_best_thresh = thresholds[np.argmax(tpr - fpr)]  
print(f"Optimal threshold: {catb_best_thresh:.3f}")  
print('Metrics for Validation data - CatBoost:')  
  
catb_valid_auc, catb_valid_accuracy, catb_valid_recall, \  
catb_valid_precision, catb_valid_f1, catb_valid_specificity = print_report(y_valid, catb_preds, catb_best_thresh)
```

✓ 2.0s

```
Optimal threshold: 0.117  
Metrics for Validation data - CatBoost:  
AUC:0.655  
Accuracy:0.646  
recall:0.567  
precision:0.172  
F1 Score:0.264  
specificity:0.656  
prevalence:0.112
```

Hypertuned Model Results

Decision Tree - Hyper Parameter Tuning

```
recall_scoring = make_scorer(recall_score)

dc_grid = {
    'max_features':    ['sqrt', 'log2'],
    'max_depth':       range(1, 11, 1),
    'min_samples_split': range(2, 10, 2),
    'criterion':        ['gini', 'entropy']
}

dc_random = RandomizedSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_distributions=dc_grid,
    n_iter=20, cv=2, scoring=recall_scoring,
    verbose=1, random_state=111
)

dc_random.fit(x_train_res, y_train_res)

print(dc_random.best_params_)

dc_hp_preds = dc_random.best_estimator_.predict(x_valid)
dc_hp_preds_proba = dc_random.best_estimator_.predict_proba(x_valid)[:, 1]

fpr, tpr, thresholds = roc_curve(y_valid, dc_hp_preds_proba)
dc_hp_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {dc_hp_best_thresh:.3f}")

dc_hp_auc, dc_hp_accuracy, dc_hp_recall, \
    dc_hp_precision, dc_hp_f1, dc_hp_specificity = print_report(y_valid, dc_hp_preds_proba, dc_hp_best_thresh)
```

✓ 4.6s

Fitting 2 folds for each of 20 candidates, totalling 40 fits
{'min_samples_split': 2, 'max_features': 'log2', 'max_depth': 7, 'criterion': 'entropy'}
Optimal threshold: 0.520
AUC:0.533
Accuracy:0.523
recall:0.540
precision:0.125
F1 Score:0.203
specificity:0.520
prevalence:0.112

Random Forest - Hypertuning

```
rf_grid = {
    'n_estimators':    range(200, 1000, 200),
    'max_features':     ['sqrt', 'log2'],
    'max_depth':        range(1, 11, 1),
    'min_samples_split': range(2, 10, 2),
    'criterion':        ['gini', 'entropy']
}

rf_random = RandomizedSearchCV(
    RandomForestClassifier(random_state=111, n_jobs=-1),
    param_distributions=rf_grid,
    n_iter=20, cv=2, scoring=recall_scoring,
    verbose=1, random_state=111
)

rf_random.fit(x_train_res, y_train_res)

print(rf_random.best_params_)

rf_hp_preds = rf_random.best_estimator_.predict(x_valid)
rf_hp_preds_proba = rf_random.best_estimator_.predict_proba(x_valid)[:, 1]

fpr, tpr, thresholds = roc_curve(y_valid, rf_hp_preds_proba)
rf_hp_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {rf_hp_best_thresh:.3f}")

rf_hp_auc, rf_hp_accuracy, rf_hp_recall, \
    rf_hp_precision, rf_hp_f1, rf_hp_specificity = print_report(y_valid, rf_hp_preds_proba, rf_hp_best_thresh)
```

✓ 1m 2.8s

Fitting 2 folds for each of 20 candidates, totalling 40 fits
{'n_estimators': 800, 'min_samples_split': 2, 'max_features': 'log2', 'max_depth': 10, 'criterion': 'gini'}
Optimal threshold: 0.467
AUC:0.628
Accuracy:0.660
recall:0.507
precision:0.167
F1 Score:0.251
specificity:0.679
prevalence:0.112

Hypertuned Model Results

XGBoost - Hypertuning

```
xgb_grid = {
    'min_child_weight': [1, 5, 8, 10],
    'gamma':            [0.5, 1, 1.5, 2, 5],
    'subsample':         [0.6, 0.8, 0.9, 1.0],
    'colsample_bytree':  [0.6, 0.8, 0.9, 1.0],
    'max_depth':         [3, 4, 5]
}

xgb_random = GridSearchCV(
    xgb.XGBClassifier(eval_metric='logloss', random_state=42),
    param_grid=xgb_grid,
    cv=2, scoring=recall_scoring, verbose=1
)
xgb_random.fit(x_train_res, y_train_res)

print(xgb_random.best_params_)

xgb_hp_preds      = xgb_random.best_estimator_.predict(x_valid)
xgb_hp_preds_proba = xgb_random.best_estimator_.predict_proba(x_valid)[:, 1]

fpr, tpr, thresholds = roc_curve(y_valid, xgb_hp_preds_proba)
xgb_hp_best_thresh = thresholds[np.argmax(tpr - fpr)]
print(f"Optimal threshold: {xgb_hp_best_thresh:.3f}")

xgb_hp_auc, xgb_hp_accuracy, xgb_hp_recall, \
| xgb_hp_precision, xgb_hp_f1, xgb_hp_specificity = print_report(y_valid, xgb_hp_preds_proba, xgb_hp_best_thresh)
✓ 19m 40.1s

Fitting 2 folds for each of 960 candidates, totalling 1920 fits
{'colsample_bytree': 1.0, 'gamma': 1.5, 'max_depth': 5, 'min_child_weight': 1, 'subsample': 0.9}
Optimal threshold: 0.142
AUC:0.662
Accuracy:0.665
recall:0.554
precision:0.179
F1 Score:0.271
specificity:0.679
prevalence:0.112
```

Hypertuned Model Results

CatBoost - Hypertuning

```
# Calculate scale_pos_weight for class imbalance
neg_count = (y_train_res == 0).sum()
pos_count = (y_train_res == 1).sum()
base_scale_weight = neg_count / pos_count
print(f"Calculated scale_pos_weight: {base_scale_weight:.2f}")

# Improved grid -- NO learning_rate above 0.1
catb_grid = {
    'iterations': [400, 600, 800], # More iterations
    'depth': [4, 5, 6], # Good range
    'learning_rate': [0.03, 0.05, 0.1], # -- FIX: Remove 1.0, keep moderate
    'l2_leaf_reg': [1, 3, 5], # Regularization to prevent overfit
    'scale_pos_weight': [base_scale_weight] # Handle class imbalance properly
}

catb_random = GridSearchCV(
    CatBoostClassifier(random_state=111, verbose=0),
    param_grid=catb_grid,
    cv=3, # Increase to 3 folds for better estimates
    scoring=recall_scoring,
    verbose=1
)
catb_random.fit(x_train_res, y_train_res)

print(f"Best params: {catb_random.best_params_}")
```

```
# Find threshold that gives recall in the TARGET RANGE (0.60-0.70)
print("\nFinding threshold for balanced recall (0.60-0.70):")
print(f"{'Threshold':<12} {'Recall':<10} {'Precision':<12} {'AUC':<8} {'F1':<8}")
print("-" * 60)
```

```
target_min_recall = 0.60
target_max_recall = 0.70
best_thresh_balanced = None
best_f1 = 0
```

```
for thresh in np.arange(0.05, 0.40, 0.01):
    rec = recall_score(y_valid, (catb_hp_preds_proba >= thresh))
    prec = precision_score(y_valid, (catb_hp_preds_proba >= thresh))
    auc = roc_auc_score(y_valid, catb_hp_preds_proba)
```

```
# Calculate F1 score as a balance metric
if prec + rec > 0:
    f1 = 2 * (prec * rec) / (prec + rec)
else:
    f1 = 0
```

```
# Find threshold in target range with best F1
if target_min_recall <= rec <= target_max_recall:
    print(f"{'thresh':<12.3f} {'rec':<10.3f} {'prec':<12.3f} {'auc':<8.3f} {'f1':<8.3f}")
    if f1 > best_f1:
        best_f1 = f1
        best_thresh_balanced = thresh
```

CatBoost Tuned - Final Results (Balanced)

```
AUC:0.670
Accuracy:0.592
recall:0.644
precision:0.164
F1 Score:0.262
specificity:0.585
prevalence:0.112
Threshold: 0.110
```


Hypertuned Model -

RandomizedSearchCV & GridSearchCV

```
dc_random = RandomizedSearchCV(  
    DecisionTreeClassifier(random_state=42),  
    param_distributions=dc_grid,  
    n_iter=20, cv=2, scoring=recall_scoring,  
    verbose=1, random_state=111  
)  
dc_random.fit(x_train_res, y_train_res)
```

```
xgb_random = GridSearchCV(  
    xgb.XGBClassifier(eval_metric='logloss', random_state=42),  
    param_grid=xgb_grid,  
    cv=2, scoring=recall_scoring, verbose=1  
)  
xgb_random.fit(x_train_res, y_train_res)
```

```
rf_random = RandomizedSearchCV(  
    RandomForestClassifier(random_state=111, n_jobs=-1),  
    param_distributions=rf_grid,  
    n_iter=20, cv=2, scoring=recall_scoring,  
    verbose=1, random_state=111  
)  
rf_random.fit(x_train_res, y_train_res)
```

```
catb_random = GridSearchCV(  
    ... CatBoostClassifier(random_state=111, verbose=0),  
    ... param_grid=catb_grid,  
    ... cv=3, ... # Increase to 3 folds for better estimates  
    ... scoring=recall_scoring,  
    ... verbose=1  
)  
catb_random.fit(x_train_res, y_train_res)
```

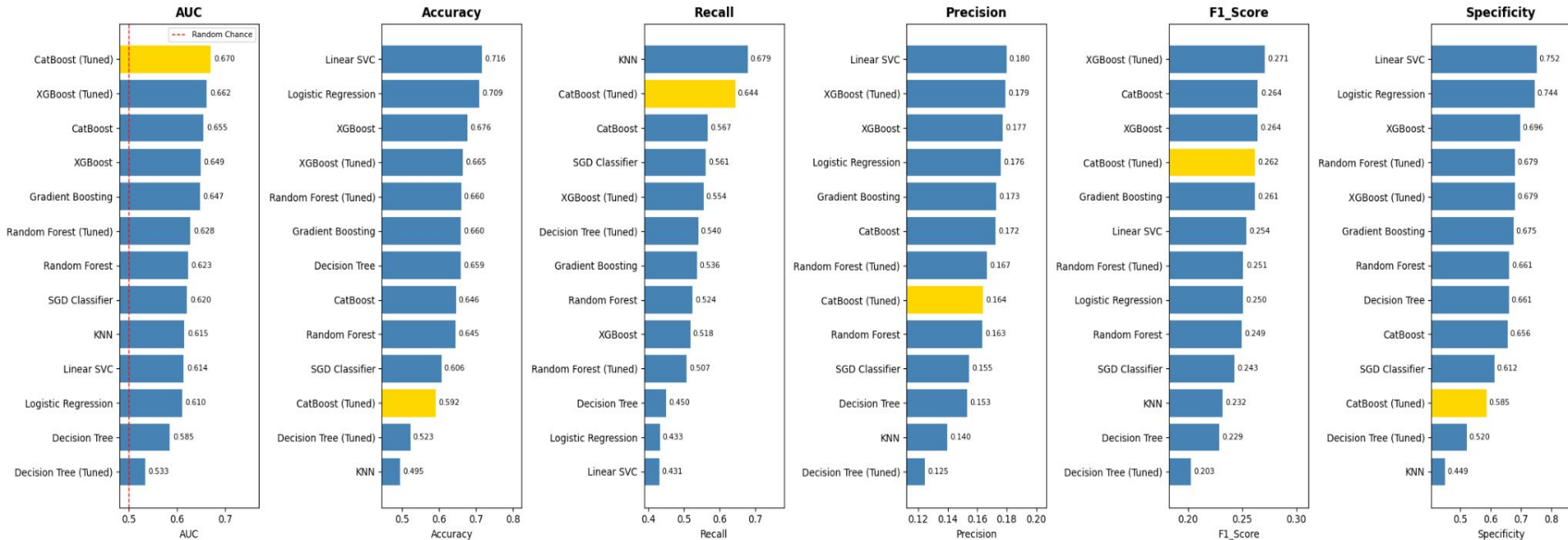
Full 13 - Model Selection Results

```
=====
MODEL COMPARISON – WITH PAYER_CODE CONFIGURATION
=====
```

Model	AUC	Accuracy	Recall	Precision	F1_Score	Specificity	Prevalence
CatBoost (Tuned)	0.669951	0.591737	0.643926	0.164074	0.261514	0.585137	0.112261
XGBoost (Tuned)	0.661634	0.664830	0.554159	0.179110	0.270721	0.678825	0.112261
CatBoost	0.654712	0.645818	0.566727	0.172338	0.264304	0.655820	0.112261
XGBoost	0.648684	0.676318	0.517654	0.177363	0.264203	0.696383	0.112261
Gradient Boosting	0.647152	0.659792	0.536206	0.172806	0.261377	0.675420	0.112261
Random Forest (Tuned)	0.627599	0.659859	0.507481	0.166667	0.250925	0.679128	0.112261
Random Forest	0.622590	0.645415	0.524237	0.163463	0.249218	0.660739	0.112261
SGD Classifier	0.620186	0.606449	0.561341	0.154709	0.242565	0.612154	0.112261
KNN	0.614703	0.495129	0.678636	0.139793	0.231831	0.449372	0.112261
Linear SVC	0.613749	0.715620	0.430880	0.179910	0.253834	0.751627	0.112261
Logistic Regression	0.610110	0.709237	0.432675	0.176213	0.250433	0.744211	0.112261
Decision Tree	0.584959	0.659389	0.450030	0.153375	0.228780	0.660512	0.112261
Decision Tree (Tuned)	0.533418	0.522741	0.539797	0.124637	0.202515	0.520357	0.112261

Full 13 - Model Selection Results

Full Model Comparison — All Metrics
Gold = CatBoost (Tuned) — Selected Final Model



Final Model Evaluation on Test Set - CatBoost (Tuned)

CatBoost Tuned - Final Results (Balanced)

AUC:0.670

Accuracy:0.592

recall:0.644

precision:0.164

F1 Score:0.262

specificity:0.585

prevalence:0.112

Threshold: 0.110

=====

FINAL TEST SET RESULTS - CatBoost (Tuned)

=====

AUC:0.678

Accuracy:0.598

recall:0.648

precision:0.167

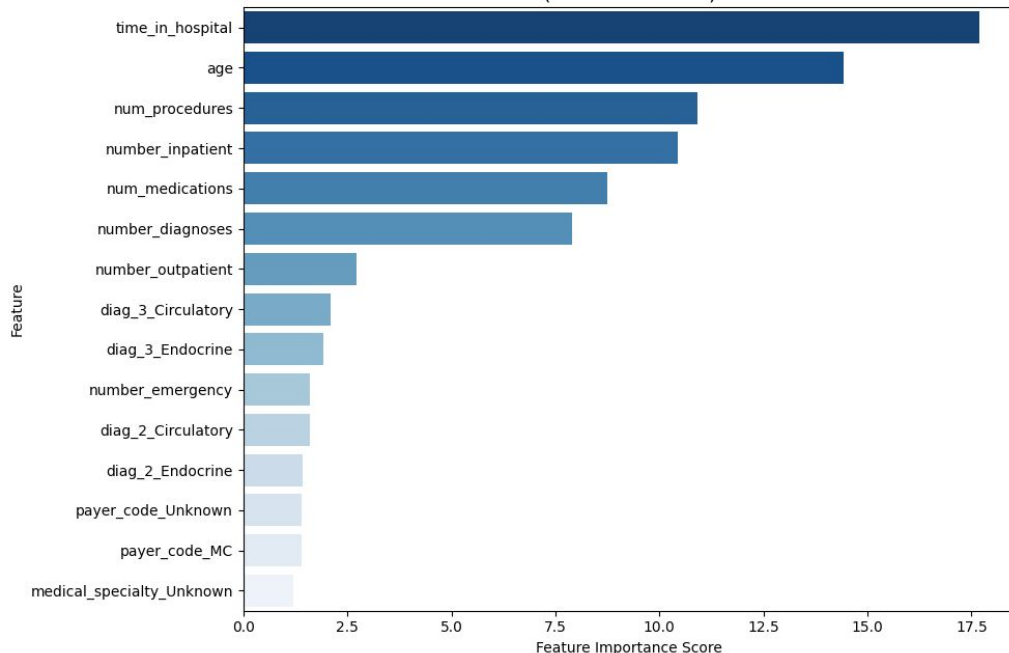
F1 Score:0.266

specificity:0.592

prevalence:0.112

Top 15 Features Predicting 30-Day Readmission CatBoost (Tuned)

Top 15 Features Predicting 30-Day Readmission
(CatBoost Tuned)

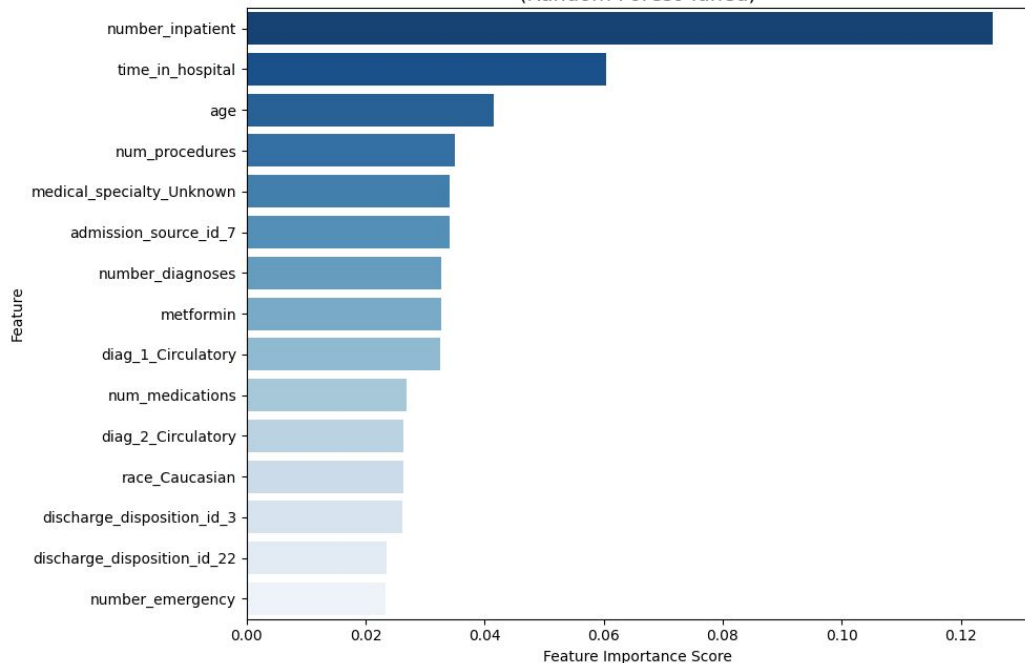


Leading Variables of Readmission

```
feat_imp = pd.DataFrame({  
    'feature': x_train.columns,  
    'importance': catb_random.best_estimator_.feature_importances_  
}).sort_values('importance', ascending=False).head(20)
```

Top 15 Features Predicting 30-Day Readmission Random Forest (Tuned) for Validation Check

Top 15 Features Predicting 30-Day Readmission
(Random Forest Tuned)

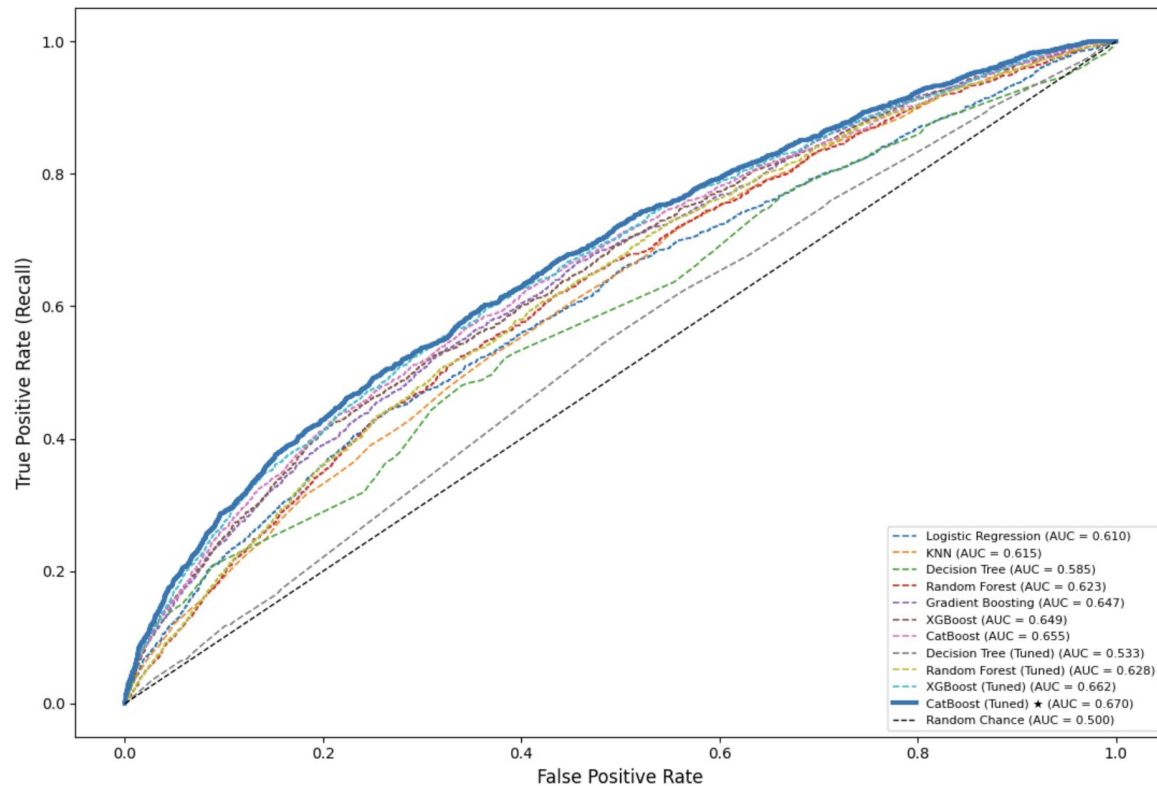


```
# — FEATURE IMPORTANCE: Random Forest (Tuned) — cross-check —  
feat_imp_rf = pd.DataFrame({  
    'feature': x_train.columns,  
    'importance': rf_random.best_estimator_.feature_importances_  
}).sort_values('importance', ascending=False).head(15)
```

All Models ROC Curve Comparison

ROC Curves — All 11 Models

★ = Selected Final Model



ROC Curve — Final Model (CatBoost Tuned)

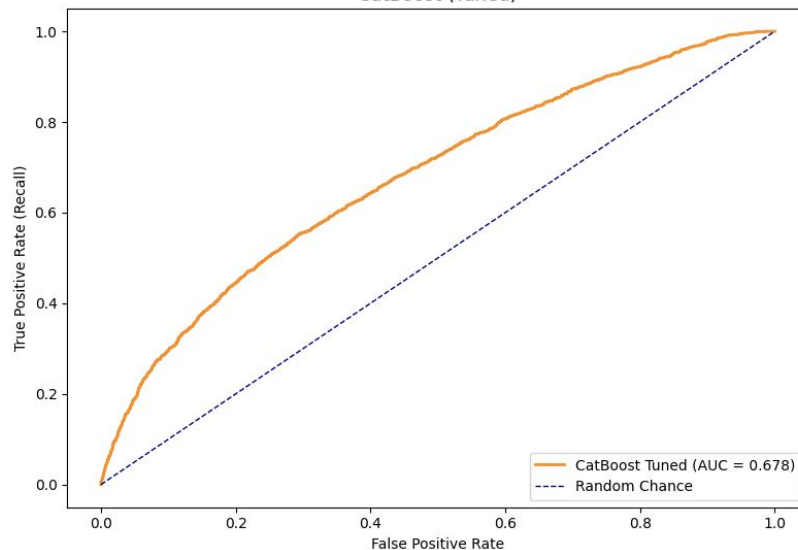
=====

FINAL TEST SET RESULTS — CatBoost (Tuned)

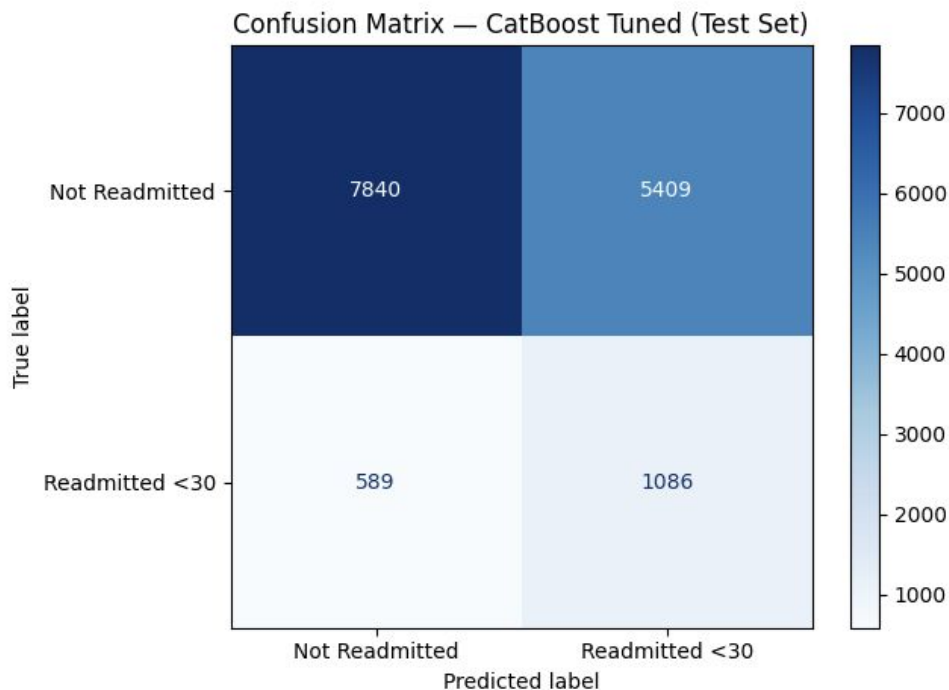
=====

AUC:0.678
Accuracy:0.598
recall:0.648
precision:0.167
F1 Score:0.266
specificity:0.592
prevalence:0.112

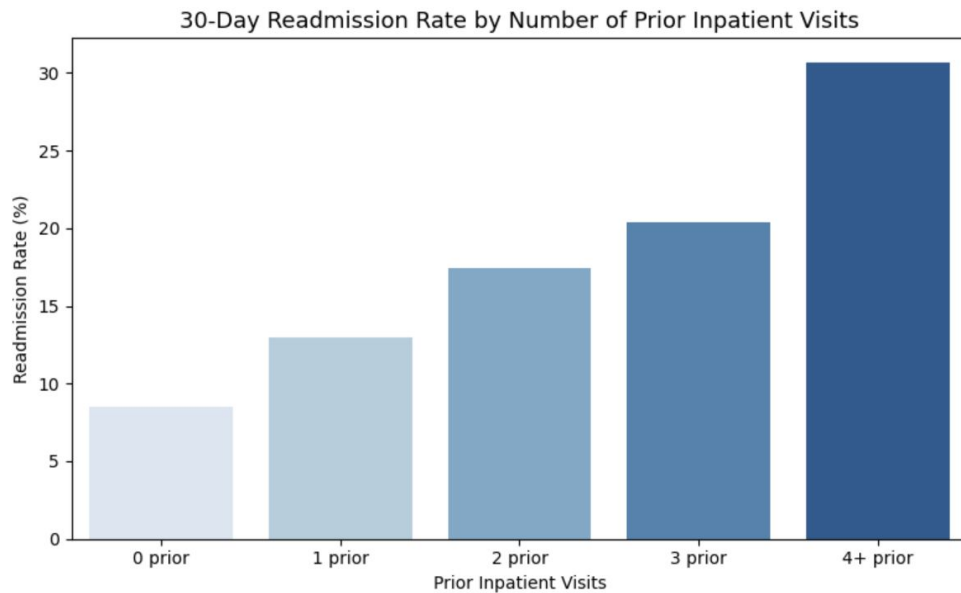
ROC Curve — Final Model on Test Set
CatBoost (Tuned)



Confusion Matrix — CatBoost Tuned

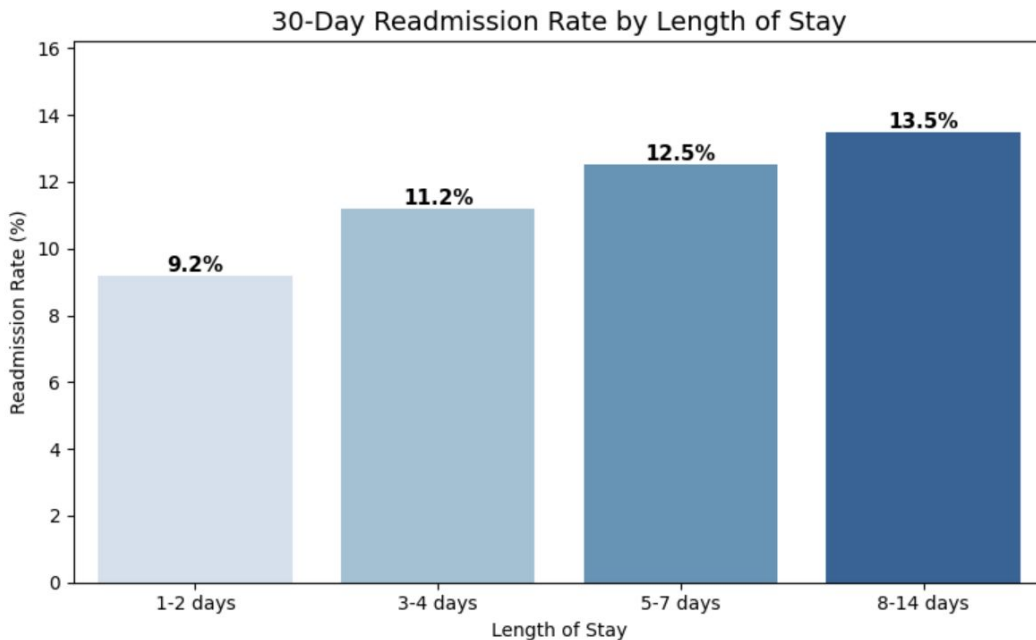


Prior Inpatient Visits vs Readmission Rate



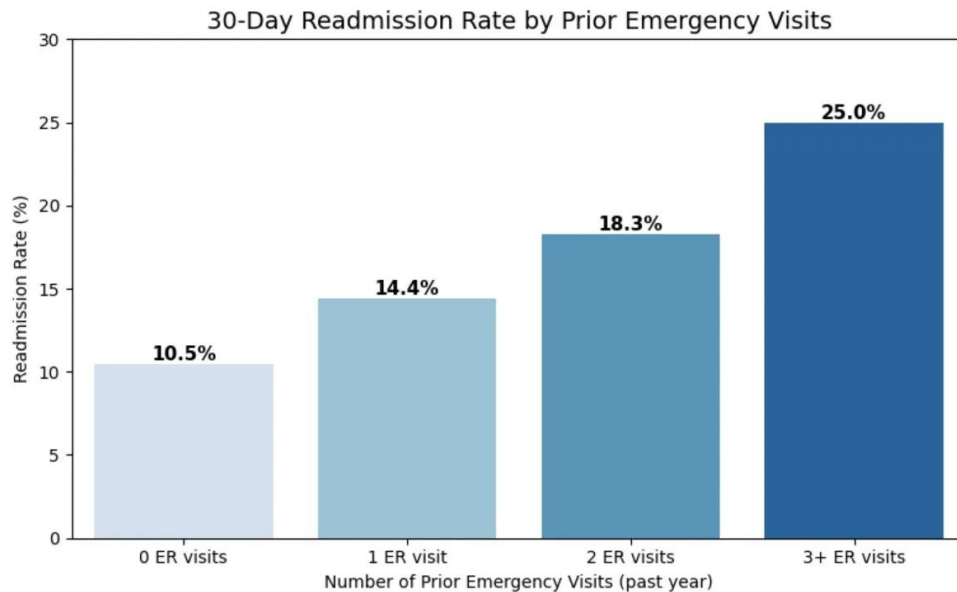
inpatient_bucket		readmission_rate_%
0	0 prior	8.5
1	1 prior	13.0
2	2 prior	17.4
3	3 prior	20.4
4	4+ prior	30.7

Length of Stay vs Readmission Rate



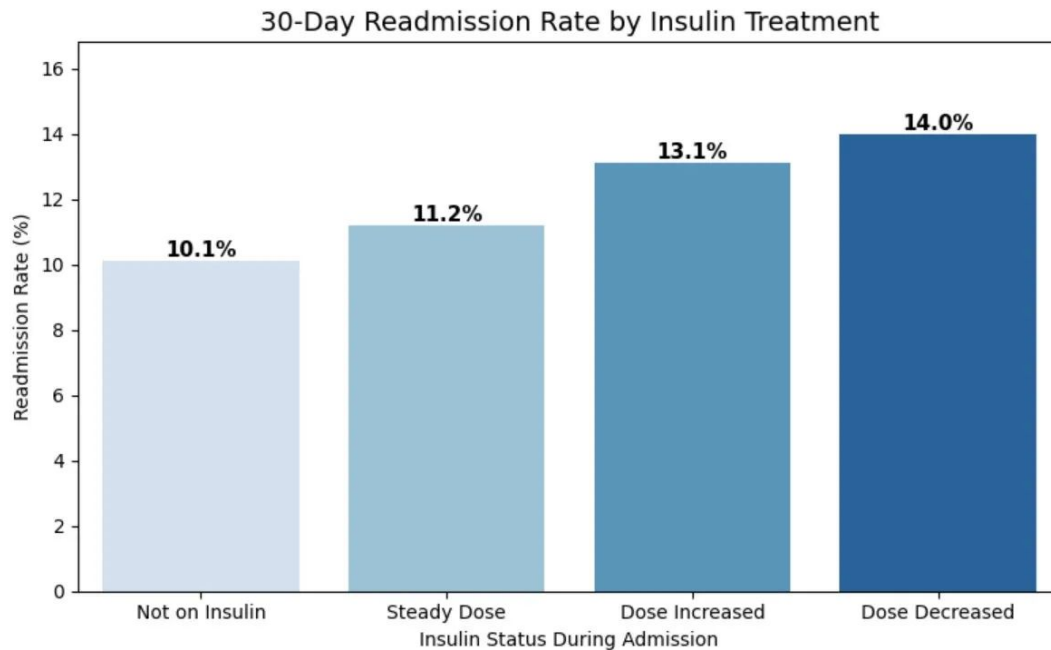
	los_bucket	readmission_rate_%
0	1-2 days	9.2
1	3-4 days	11.2
2	5-7 days	12.5
3	8-14 days	13.5

Prior Emergency Visits vs Readmission Rate



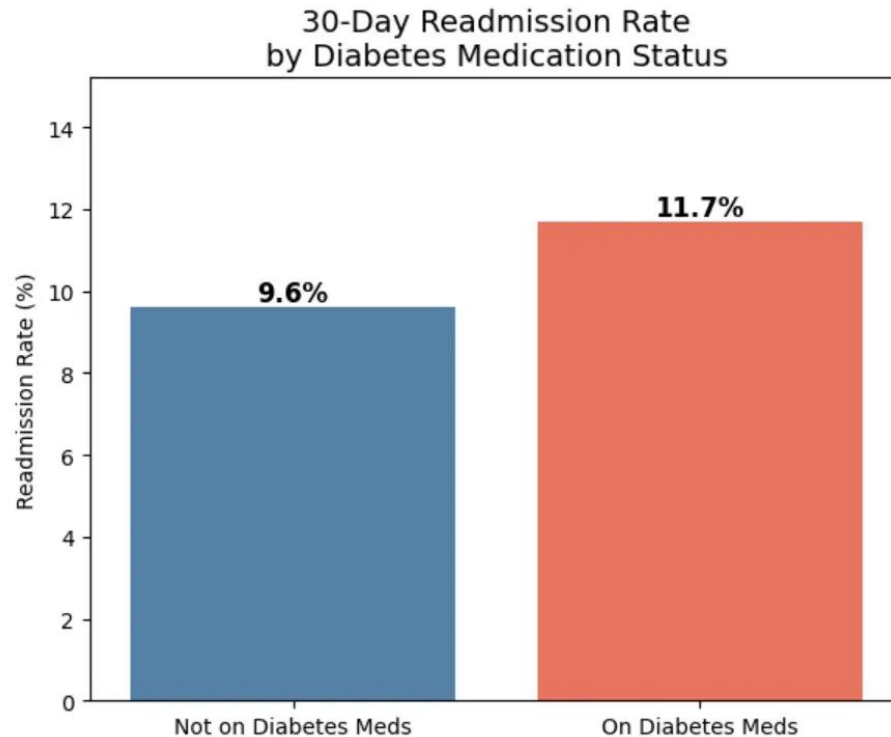
	er_bucket	readmission_rate_%
0	0 ER visits	10.5
1	1 ER visit	14.4
2	2 ER visits	18.3
3	3+ ER visits	25.0

Insulin Treatment vs Readmission Rate



	insulin_label	readmission_rate_%
0	Not on Insulin	10.1
1	Steady Dose	11.2
2	Dose Increased	13.1
3	Dose Decreased	14.0

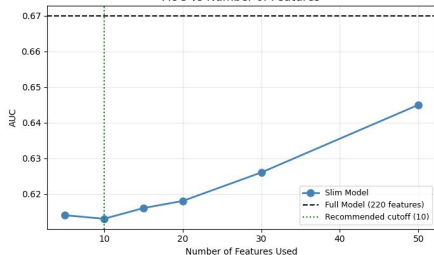
Diabetes Medication Status vs Readmission Rate



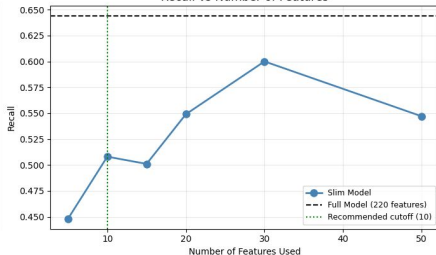
FEWEST VARIABLES ANALYSIS

Finding the Fewest Variables Needed to Predict Readmission
Goal: Smallest feature set that retains full model performance

AUC vs Number of Features



Recall vs Number of Features



Top 10 Features Selected:

1. time_in_hospital (importance: 17.679)
2. age (importance: 14.420)
3. num_procedures (importance: 10.906)
4. number_inpatient (importance: 10.454)
5. num_medications (importance: 8.756)
6. number_diagnoses (importance: 7.890)
7. number_outpatient (importance: 2.728)
8. diag_3_Circulatory (importance: 2.109)
9. diag_3_Endocrine (importance: 1.930)
10. number_emergency (importance: 1.595)

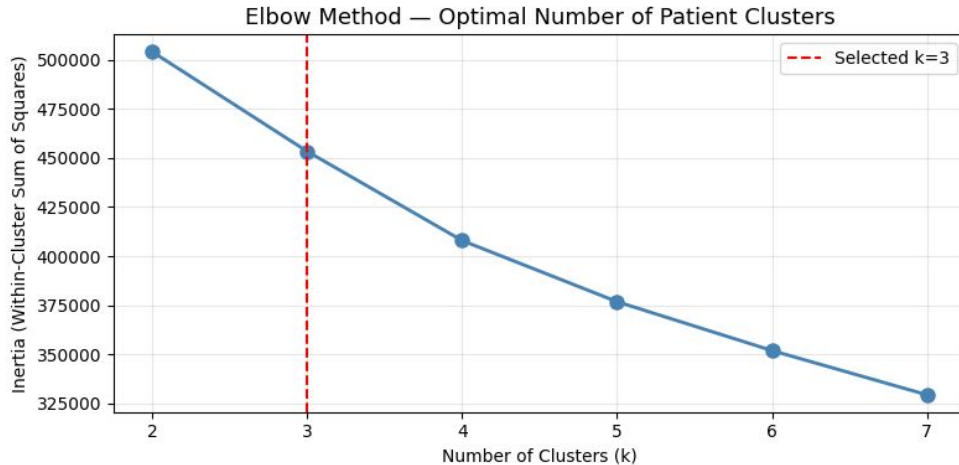
SLIM MODEL – Test Set Results (Top 10 Features)

AUC:0.621
Accuracy:0.667
recall:0.484
precision:0.165
F1 Score:0.246
specificity:0.691
prevalence:0.112

FULL vs SLIM MODEL COMPARISON (Test Set)

Metric	Full Model	Slim (10 feat)
AUC	0.670	0.621
Accuracy	0.592	0.667
Recall	0.644	0.484
Precision	0.164	0.165
F1-Score	0.262	0.246
Specificity	0.585	0.691
# Features	220	10

CLUSTERING ANALYSIS



CLUSTERING: IDENTIFYING PATIENT RISK GROUPS

Goal:

Use unsupervised learning to discover natural groupings of patients based on clinical characteristics

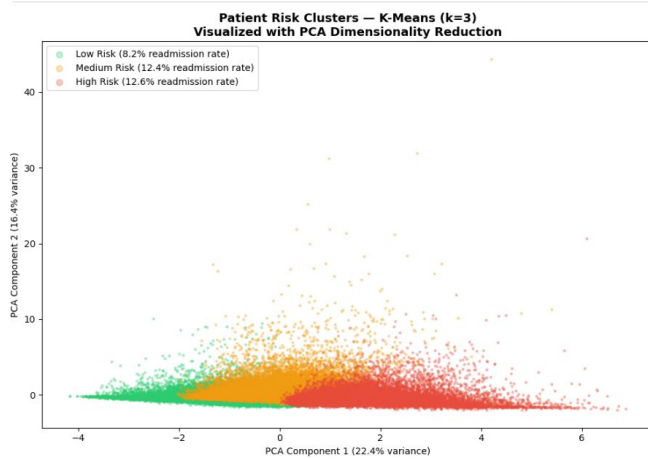
Method: K-Means Clustering

- Input: Top 10 most important features
- Algorithm: K-Means ($k=3$)
- Output: 3 distinct patient risk groups

Why This Matters:

Different patient profiles may need different interventions

CLUSTERING ANALYSIS



CLUSTER RESULTS: 3 PATIENT RISK GROUPS

Risk Group Breakdown:

- Low Risk: 20,603 patients (8.2% readmission rate)
- Medium Risk: 32,786 patients (12.4% readmission rate)
← Largest group
- High Risk: 16,295 patients (12.6% readmission rate)

Key Finding:

Medium and High risk have nearly identical readmission rates, but different clinical profiles

Medium Risk is now the largest cluster (47% of patients)

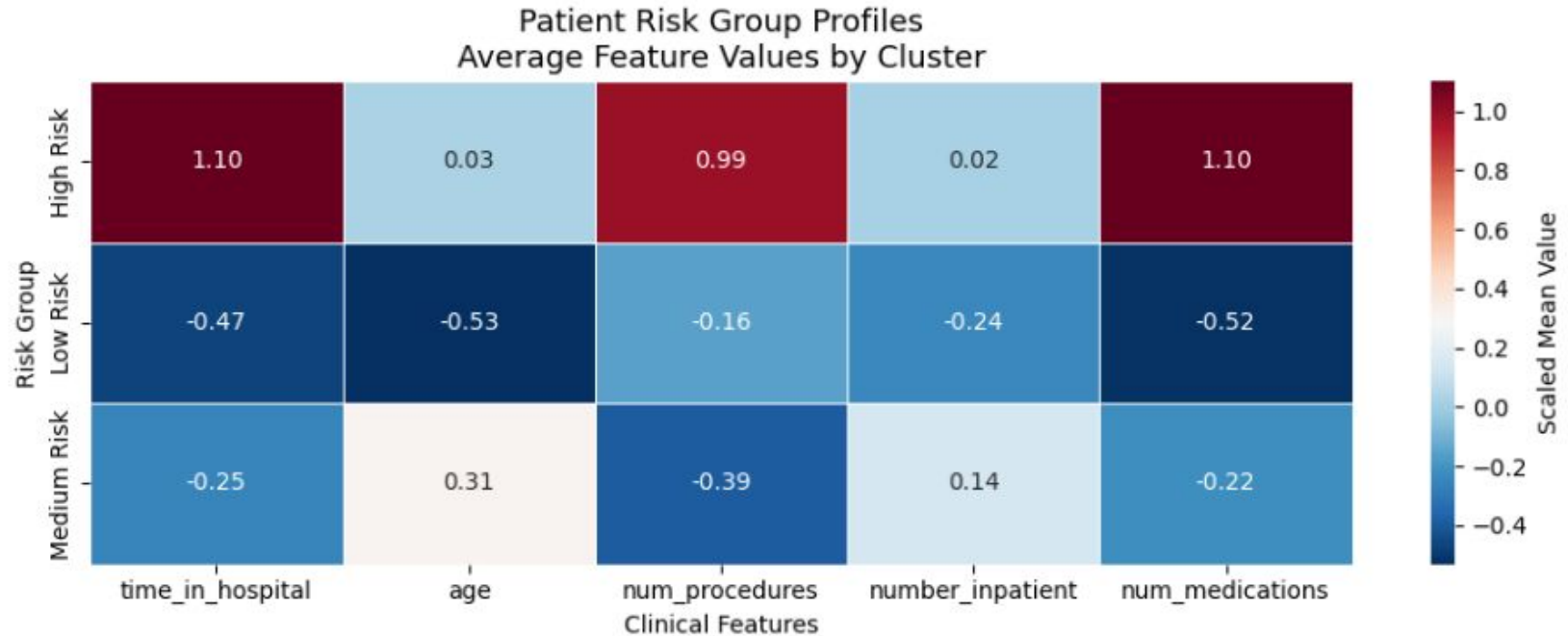
=====

CLUSTER SUMMARY — Patient Risk Groups

=====

risk_group	patient_count	readmission_rate_%
High Risk	16295	12.6
Low Risk	20603	8.2
Medium Risk	32786	12.4

CLUSTERING ANALYSIS



MULTILEVEL LINEAR MODEL

```
MLM dataset shape: (99493, 8)
```

```
Sample counts by age group:
```

```
age_group
Young (0-30)      2453
Middle (40-60)    13164
Senior (60-70)    38883
Elderly (70+)     44993
Name: count, dtype: int64
```

Mixed Linear Model Regression Results						
=====						
Model:	MixedLM	Dependent Variable: readmitted				
No. Observations:	99493	Method:	REML			
No. Groups:	4	Scale:	0.0967			
Min. group size:	2453	Log-Likelihood:	-25001.2975			
Max. group size:	44993	Converged:	No			
Mean group size:	24873.2					

	Coef.	Std.Err.	z	P> z	[0.025 0.975]	

Intercept	0.039	0.151	0.261	0.794	-0.256	0.335
number_inpatient	0.039	0.001	47.786	0.000	0.037	0.040
time_in_hospital	0.002	0.000	5.560	0.000	0.001	0.003
number_diagnoses	0.004	0.001	6.363	0.000	0.002	0.005
num_medications	0.001	0.000	4.068	0.000	0.000	0.001
number_emergency	0.006	0.001	5.903	0.000	0.004	0.009
Group Var	0.091					
=====						

MULTILEVEL LINEAR MODEL

What Is It?

A statistical model that accounts for hierarchical data structure

The Hierarchy in Our Data:

Patients are NESTED WITHIN age groups

Level 1: Individual patients

↓ (grouped by)

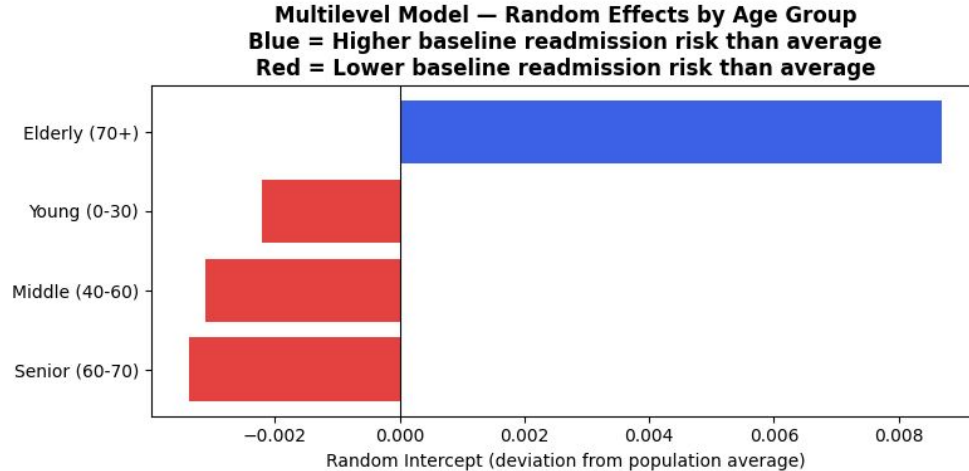
Level 2: Age groups (Young, Middle, Senior, Elderly)

Why This Matters:

Patients in the same age group share baseline risk characteristics.

Standard regression assumes all patients are independent, which is wrong.

MULTILEVEL LINEAR MODEL



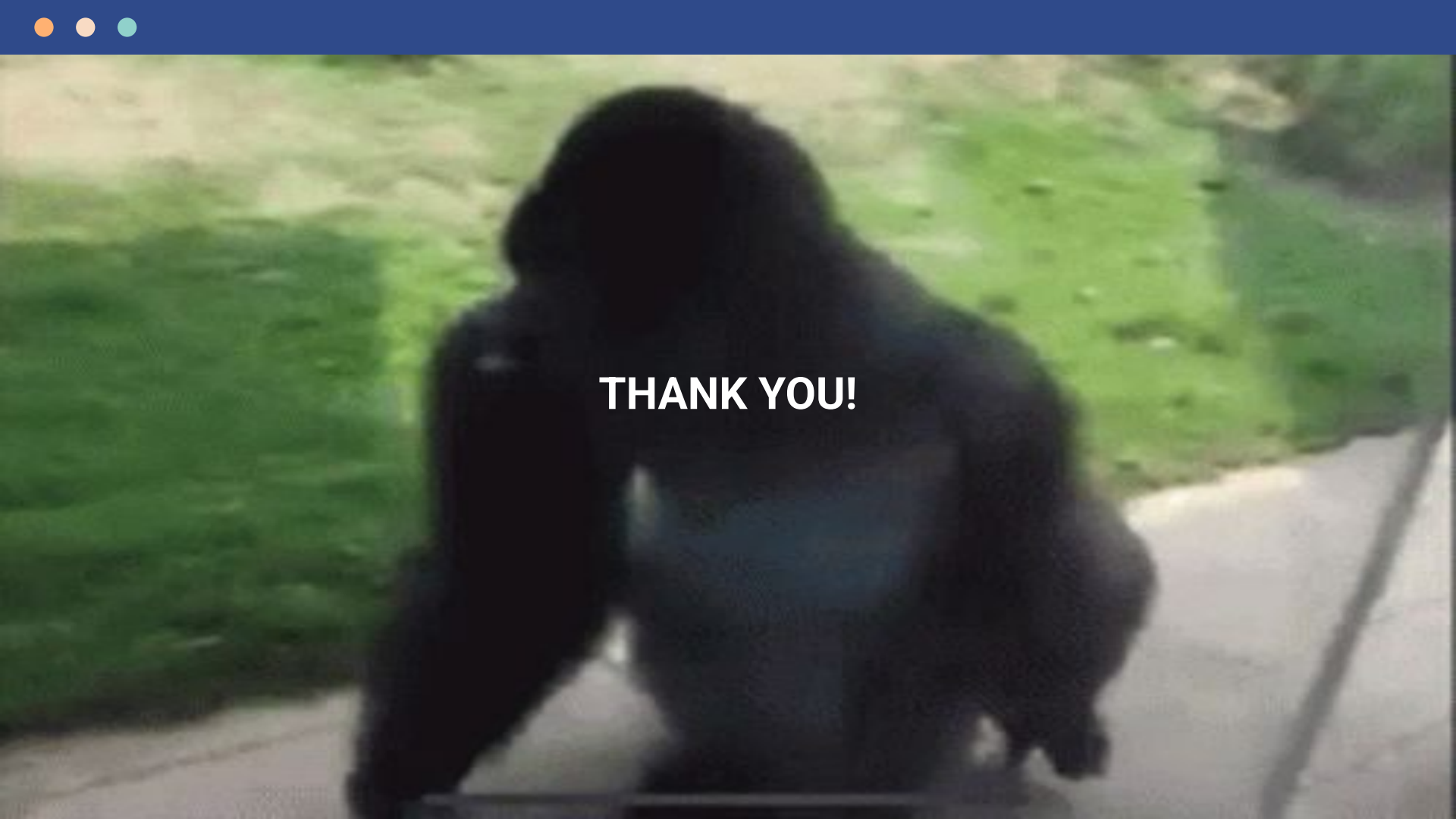
RANDOM EFFECTS: AGE GROUP DIFFERENCES

Baseline Readmission Risk by Age Group:
(controlling for all other factors)

- Elderly (70+): +0.30 (highest baseline risk)
- Senior (60-70): +0.15
- Middle (40-60): -0.10
- Young (0-30): -0.35 (lowest baseline risk)

Key Finding:

Even after accounting for prior admissions, diagnoses, etc., elderly patients have inherently higher readmission risk



THANK YOU!